EE 322
Embedded Systems Design

Lecture Notes

Department of Electrical and Electronic Engineering
Faculty of Engineering
University of Peradeniya, Sri Lanka

3 Credits | 27 Lecture Hours | 9 Modules

Academic Year 2024/2025

These notes are prepared as a comprehensive reference for the EE322 course.
They cover all nine modules of the syllabus from introduction to modeling.

Contents

1 Introduction to Embedded Systems **1**
1.1 What is an Embedded System? . . . 1
1.1.1 Contrast with General-Purpose Computers . . . 1
1.1.2 Real-Time Constraints . . . 2
1.1.3 Resource Constraints . . . 2
1.2 Application Domains . . . 3
1.2.1 Consumer Electronics . . . 3
1.2.2 Automotive . . . 3
1.2.3 Industrial Automation . . . 3
1.2.4 Medical Devices . . . 3
1.2.5 Aerospace and Defence . . . 3
1.3 Embedded System Platforms . . . 4
1.3.1 Microprocessor-Based Systems . . . 4
1.3.2 Microcontroller-Based Systems . . . 4
1.3.3 Single Board Computers (SBCs) . . . 4
1.3.4 Industrial Embedded Computers . . . 4
1.4 Design Challenges . . . 5
1.4.1 Power Consumption . . . 5
1.4.2 Area and Cost . . . 5
1.4.3 Real-Time Response . . . 5
1.4.4 Reliability and Safety . . . 6
1.4.5 Security . . . 6
1.4.6 Development Complexity . . . 6
1.5 Case Study: Anti-Lock Braking System (ABS) . . . 6
1.6 Worked Examples . . . 7
1.7 Practice Problems . . . 9

2 Microprocessor/Microcontroller Architecture **11**
2.1 A Representative Microprocessor Architecture . . . 11
2.1.1 Von Neumann vs. Harvard Architecture . . . 11
2.1.2 Core Components . . . 13
2.1.3 System Buses . . . 13
2.2 Instruction Set Architectures (ISA) . . . 14
2.2.1 CISC vs. RISC Philosophy . . . 14
2.2.2 Instruction Formats . . . 14

2.2.3 Addressing Modes . . . 15
2.2.4 ARM ISA Overview . . . 15
2.3 Instruction Fetch and Execution Cycles . . . 15
2.3.1 Single-Cycle and Multi-Cycle Execution . . . 15
2.3.2 Pipelining . . . 16
2.3.3 Pipeline Hazards . . . 16
2.3.4 Branch Prediction . . . 17
2.4 Microcontroller Internals . . . 18
2.4.1 On-Chip Peripherals . . . 18
2.4.2 Memory Map of a Typical MCU . . . 19
2.4.3 Interrupt Handling . . . 19
2.5 ARM Assembly Code Examples . . . 20
2.5.1 Data Processing Instructions . . . 20
2.5.2 Load/Store and Branch Instructions . . . 20
2.5.3 C Code with Inline Assembly . . . 21
2.6 Worked Examples . . . 22
2.7 Practice Problems . . . 24

3 Embedded Processors **26**
3.1 Types of Embedded Processors . . . 26
3.1.1 General-Purpose Microcontrollers . . . 26
3.1.2 Digital Signal Processors (DSPs) . . . 26
3.1.3 Application Processors . . . 27
3.1.4 FPGAs and Reconfigurable Computing . . . 27
3.1.5 ASICs (Application-Specific Integrated Circuits) . . . 28
3.2 ARM Processor Family . . . 28
3.2.1 Cortex-M: Microcontroller Profile . . . 28
3.2.2 Cortex-R: Real-Time Profile . . . 28
3.2.3 Cortex-A: Application Profile . . . 28
3.2.4 Thumb-2 ISA Deep Dive . . . 29
3.2.5 CMSIS Standard . . . 29
3.3 SHARC Processors (Analog Devices) . . . 30
3.4 Embedded Systems Programming . . . 31
3.4.1 Bare-Metal C Programming Idioms . . . 31
3.4.2 Startup Code and Linker Scripts . . . 32
3.4.3 Memory Sections . . . 34
3.4.4 Cross-Compilation Toolchain . . . 34
3.5 Parallelism in Embedded Systems . . . 35
3.5.1 Instruction-Level Parallelism (ILP) . . . 35
3.5.2 Task-Level Parallelism . . . 35
3.5.3 Hardware Parallelism . . . 35
3.5.4 SIMD for Embedded: ARM NEON . . . 35
3.6 Worked Examples . . . 36
3.7 Practice Problems . . . 38

4 Memory Architectures **40**
4.1 Memory Technologies . . . 40
4.1.1 SRAM (Static Random-Access Memory) . . . 40
4.1.2 DRAM (Dynamic Random-Access Memory) . . . 41
4.1.3 Flash Memory . . . 41
4.1.4 EEPROM . . . 41
4.1.5 ROM Variants . . . 41
4.1.6 Emerging Non-Volatile Memories . . . 41
4.2 Memory Hierarchy . . . 42
4.2.1 Locality of Reference . . . 42
4.2.2 Cache Operation . . . 42
4.2.3 Cache Coherency in Multicore Systems . . . 43
4.3 Memory Models and Mapping . . . 43
4.3.1 Physical vs. Virtual Memory . . . 43
4.3.2 Memory-Mapped I/O . . . 44
4.3.3 DMA (Direct Memory Access) . . . 44
4.4 Worked Examples . . . 45
4.5 Practice Problems . . . 47

5 I/O Ports **49**
5.1 I/O Hardware . . . 49
5.1.1 Digital GPIO . . . 49
5.1.2 Analog I/O . . . 50
5.1.3 Signal Conditioning . . . 50
5.2 Microprocessor/Microcontroller Interfacing . . . 51
5.2.1 Polling vs. Interrupt-Driven I/O vs. DMA . . . 51
5.2.2 ISR Design Rules . . . 51
5.2.3 GPIO Register Programming . . . 51
5.3 Analog and Digital Interfaces . . . 52
5.3.1 ADC: Resolution and Sampling . . . 52
5.3.2 PWM (Pulse Width Modulation) . . . 52
5.4 Sequential Interfaces . . . 53
5.4.1 UART (Universal Asynchronous Receiver/Transmitter) . . . 53
5.4.2 SPI (Serial Peripheral Interface) . . . 53
5.4.3 I^2C (Inter-Integrated Circuit) . . . 54
5.4.4 Other Buses . . . 54
5.5 Code Examples . . . 55
5.6 Worked Examples . . . 56
5.7 Practice Problems . . . 58

6 Multitasking **59**
6.1 Imperative Programs — Single Task Baseline . . . 59
6.1.1 Super-Loop Architecture . . . 59
6.1.2 State Machine Approach . . . 60
6.2 Processes and Threads . . . 60

6.2.1 Thread States . . . 61
6.3 Context Switching Mechanics . . . 62
6.3.1 What Is Saved . . . 62
6.3.2 ARM Cortex-M: PSP vs. MSP . . . 62
6.3.3 PendSV Context Switch . . . 62
6.3.4 Context Switch Overhead . . . 63
6.4 Inter-Task Communication . . . 63
6.4.1 Shared Memory with Mutexes . . . 63
6.4.2 Message Passing: Queues and Mailboxes . . . 64
6.4.3 Semaphores . . . 65
6.4.4 Event Flags / Event Groups . . . 65
6.5 FreeRTOS as a Concrete Example . . . 65
6.6 Worked Examples . . . 66
6.7 Practice Problems . . . 68

7 Task Scheduling 69
7.1 Scheduling Fundamentals . . . 69
7.1.1 Task Model . . . 69
7.1.2 Classification of Schedulers . . . 70
7.2 Rate Monotonic Scheduling (RMS) . . . 70
7.2.1 RMS Utilisation Bound . . . 70
7.2.2 Response Time Analysis (RTA) . . . 71
7.3 Earliest Deadline First (EDF) . . . 71
7.3.1 Comparison: RMS vs. EDF . . . 72
7.4 Deadline Monotonic Scheduling (DMS) . . . 72
7.5 Gantt Charts . . . 72
7.6 Multiprocessor Scheduling . . . 75
7.7 Worst-Case Execution Time (WCET) . . . 75
7.8 Practice Problems . . . 76

8 Analysis and Verification of Embedded Systems 78
8.1 Performance Analysis and Optimisation . . . 78
8.1.1 Profiling Metrics . . . 78
8.1.2 ARM Cortex-M DWT Cycle Counter . . . 78
8.1.3 Memory Analysis . . . 79
8.1.4 Optimisation Techniques . . . 80
8.2 Failure Mode and Effects Analysis (FMEA) . . . 80
8.3 Reliability and Availability . . . 81
8.3.1 Bathtub Curve . . . 82
8.3.2 Redundancy: Triple Modular Redundancy (TMR) . . . 82
8.4 Formal Verification . . . 82
8.4.1 Model Checking with SPIN and Promela . . . 83
8.4.2 Temporal Logic . . . 84
8.4.3 Timed Automata and UPPAAL . . . 84
8.5 MISRA-C Guidelines . . . 84

8.5.1 CRC for Data Integrity . . . 85
8.6 Worked Examples . . . 86
8.7 Practice Problems . . . 87

9 Modeling of Embedded Systems 89
9.1 UML for Embedded Systems . . . 89
9.1.1 UML Class Diagram . . . 90
9.1.2 UML Sequence Diagram . . . 90
9.2 Finite State Machines (FSMs) . . . 91
9.2.1 Mealy vs. Moore Machines . . . 91
9.2.2 Example: Traffic Light Controller FSM . . . 91
9.3 FSM Implementation in C . . . 91
9.3.1 Switch-Case Implementation . . . 91
9.3.2 Function Pointer Table Implementation . . . 93
9.4 Hierarchical State Machines (HSMs) . . . 94
9.4.1 Statechart Example: Motor Controller . . . 95
9.4.2 HSM Implementation Approach . . . 95
9.5 State Flow Models . . . 97
9.5.1 Stateflow in Simulink . . . 97
9.5.2 Model-Driven Development Workflow . . . 97
9.6 Worked Examples . . . 97
9.7 Practice Problems . . . 101

References 103

List of Figures

1.1 Generic embedded system block diagram: sensor → MCU → actuator with power supply and communication interface. 7

2.1 Von Neumann architecture: single shared bus for instructions and data. . . 12
2.2 Harvard architecture: separate instruction and data buses enabling simultaneous access. 12
2.3 5-stage instruction pipeline showing overlapped execution of four instructions. 16
2.4 2-bit saturating counter branch predictor state machine. 17
2.5 ARM Cortex-M3/M4 3-stage pipeline architecture. 17
2.6 Simplified memory map of an STM32F4 microcontroller (Cortex-M4). . . . 19
2.7 Interrupt latency: from IRQ assertion to first ISR instruction on ARM Cortex-M3. 20
2.8 5-stage pipeline datapath showing EX-to-EX and MEM-to-EX forwarding paths (dashed red arrows) that resolve data hazards without stalling. . . . 21

3.1 SHARC super-Harvard architecture with dual data memory buses enabling three parallel memory accesses. 30

4.1 Simplified 6T SRAM cell: two cross-coupled inverters with two access transistors controlled by the word line. 40
4.2 Memory hierarchy pyramid: speed and cost increase towards the top; capacity increases towards the bottom. 42
4.3 STM32F407 memory map showing Flash, SRAM, peripheral, and Cortex-M system address regions. 44

5.1 SAR ADC block diagram: successive approximation register with internal DAC feedback. 50
5.2 UART frame format: idle HIGH, start bit (LOW), 8 data bits (LSB first), optional parity, stop bit (HIGH). 53
5.3 SPI Mode 0 timing: CPOL=0, CPHA=0. Data sampled on rising clock edge. 54
5.4 I^2C transaction: START (S), 7-bit address, R/W bit, ACK from slave, STOP (P). 54

6.1 Thread (task) state transition diagram: New → Ready → Running ↔ Blocked → Terminated. 61

6.2 Priority inversion: high-priority task H is blocked while medium-priority task M runs. 64
6.3 Priority inheritance: L inherits H's priority, preventing M from causing unbounded inversion. 64
6.4 Producer-consumer pattern using a FIFO queue for inter-task communication. 65

7.1 RMS Gantt chart for the task set over the hyperperiod $[0, 12)$. 73
7.2 EDF schedule for the same task set. Note the different execution order due to dynamic priorities. 74

8.1 Bathtub curve: failure rate over the lifetime of electronic systems. 82
8.2 Triple Modular Redundancy: three identical modules feed a majority voter. 82

9.1 UML class diagram: temperature controller with sensor abstraction. 90
9.2 UML sequence diagram: periodic temperature control loop. 90
9.3 FSM for a traffic light controller with emergency override. 91
9.4 HSM: Motor controller with "Operational" superstate and "Fault" handling. The fault transition applies to all substates simultaneously. 95
9.5 Vending machine FSM. 98
9.6 Microwave oven UML state machine diagram. 99

List of Tables

1.1 General-purpose computer vs. embedded system. 2
1.2 Embedded system application domains and constraints. 3
1.3 Platform comparison: microprocessor vs. microcontroller vs. SBC vs. industrial PC. 5

2.1 Von Neumann vs. Harvard architecture comparison. 13
2.2 CISC vs. RISC design philosophies. 14
2.3 Common addressing modes in embedded processors. 15

3.1 ARM Cortex family comparison matrix. 29
3.2 Standard memory sections in an embedded C program. 34

4.1 Memory technology comparison. 42

5.1 I/O strategies comparison. 51
5.2 SPI modes. 54

7.1 Classification of scheduling algorithms. 70
7.2 Liu–Layland utilisation bound $U_{\text{lub}}(n) = n(2^{1/n} - 1)$. 70
7.3 Comparison of RMS and EDF scheduling algorithms. 72

8.1 Common embedded optimisation strategies. 80
8.2 Example FMEA table for an embedded temperature controller. 81
8.3 Selected MISRA-C 2012 rules relevant to embedded programming. 85

Chapter 1

Introduction to Embedded Systems

Module: 1 **Lecture Hours:** 2 **ILOs Addressed:** ILO-1, ILO-2

Learning Outcomes: By the end of this module, students will be able to:

1. Define an embedded system and distinguish it from a general-purpose computer.
2. Identify application domains and classify systems by their real-time and safety requirements.
3. Compare embedded system platforms (microprocessor, microcontroller, SBC, industrial PC).
4. Analyse the key design challenges in embedded systems engineering.

1.1 What is an Embedded System?

Definition 1.1: Embedded System

An **embedded system** is a special-purpose computing system designed to perform one or a few dedicated functions, often as part of a larger mechanical or electrical system. Unlike general-purpose computers, embedded systems are optimised for specific tasks with constraints on power, size, cost, and real-time responsiveness.

An embedded system typically consists of a processor (microprocessor or microcontroller), memory, input/output peripherals, and application-specific software. The software, often called *firmware*, is stored in non-volatile memory (usually Flash) and executes immediately upon power-on without user intervention.

1.1.1 Contrast with General-Purpose Computers

A desktop PC or laptop is designed to run a wide variety of applications chosen by the user. In contrast, an embedded system is pre-programmed at the factory to perform a specific function. table 1.1 highlights the key differences.

Table 1.1: General-purpose computer vs. embedded system.

Attribute	**General-Purpose Computer**	**Embedded System**
Primary function	Runs user-selected applications	Dedicated single/few functions
Operating system	Full OS (Windows, Linux, macOS)	Bare-metal, RTOS, or lightweight Linux
User interface	Rich GUI, keyboard, mouse	Minimal: buttons, LEDs, LCD, or none
Processing power	High (multi-GHz, multi-core)	Low–moderate (kHz to hundreds of MHz)
Memory	GB of RAM, TB of storage	KB–MB of RAM, KB–MB of Flash
Power consumption	50–500 W	μW–W range
Cost per unit	\$500–\$5000+	\$0.10–\$50
Real-time constraints	Generally none	Often hard or soft real-time

1.1.2 Real-Time Constraints

Many embedded systems must respond to events within strict time limits. These constraints are classified as:

- **Hard real-time:** Missing a deadline leads to system failure or catastrophic consequences (e.g., airbag deployment, anti-lock braking).
- **Soft real-time:** Missing a deadline degrades quality but does not cause failure (e.g., video streaming, audio playback).
- **Non-real-time:** No timing constraints on response (e.g., a temperature logger that records data hourly).

1.1.3 Resource Constraints

Embedded systems operate under severe resource limitations compared to desktop machines:

- **Processing:** Clock frequencies from 1 MHz (low-power MCU) to 1 GHz (application processor).
- **Memory:** As little as 256 bytes of RAM on an 8-bit MCU; typically 64 KB–1 MB for Cortex-M class.
- **Power:** Battery-operated devices may need to run for years on a coin cell ($<50\,\mu$A average current).
- **Cost:** High-volume consumer products demand sub-dollar BOM costs per unit.

1.2 Application Domains

Embedded systems are ubiquitous. table 1.2 classifies major application domains by their constraints.

Table 1.2: Embedded system application domains and constraints.

Domain	RT Type	Power Budget	Safety Criticality	Volume
Consumer electronics	Soft	Low–Medium	Low	Very High
Automotive	Hard	Medium	High (ASIL D)	High
Industrial automation	Hard	Medium–High	Medium–High	Medium
Medical devices	Hard/Soft	Low–Medium	Very High (IEC 62304)	Medium
Aerospace & defence	Hard	Varies	Extreme (DO-178C)	Low
Telecommunications	Soft	High	Medium	High
IoT & wearables	Soft/None	Ultra-Low	Low	Very High

1.2.1 Consumer Electronics

Smartphones, smart TVs, game controllers, wireless earbuds, and fitness trackers all contain embedded processors. These systems prioritise low cost, low power, and user responsiveness.

1.2.2 Automotive

A modern vehicle contains 70–150 Electronic Control Units (ECUs), each an embedded system. Functions include engine management, ABS, airbag control, infotainment, and Advanced Driver Assistance Systems (ADAS). Automotive embedded systems must comply with ISO 26262 functional safety standards.

1.2.3 Industrial Automation

Programmable Logic Controllers (PLCs), motor drives, and CNC machines rely on embedded processors for precise, deterministic control loops. Communication buses like CAN, Modbus, and EtherCAT interconnect these units.

1.2.4 Medical Devices

Pacemakers, insulin pumps, patient monitors, and MRI scanners are safety-critical embedded systems. IEC 62304 governs software lifecycle processes, and high reliability (MTTF $>$ 100,000 hours) is mandatory.

1.2.5 Aerospace and Defence

Flight computers, missile guidance systems, satellite subsystems, and radar signal processors operate under extreme environmental conditions (temperature, vibration, radiation). DO-178C certification requires rigorous verification.

1.3 Embedded System Platforms

Embedded systems can be built on a range of hardware platforms, each offering different tradeoffs.

1.3.1 Microprocessor-Based Systems

A microprocessor (MPU) contains only the CPU core: ALU, control unit, and register file. External chips provide memory (SRAM, DRAM, Flash), I/O peripherals, and clock generation on a PCB. Examples: Intel x86 embedded variants, older Motorola 68000. This approach offers flexibility but increased board complexity, power, and cost.

1.3.2 Microcontroller-Based Systems

A microcontroller (MCU) integrates the CPU, Flash memory, SRAM, and peripherals (timers, ADC, UART, SPI, I^2C, GPIO) on a single chip. This integration dramatically reduces PCB complexity and cost. MCUs dominate the embedded world, with families such as:

- **8-bit:** Atmel AVR (ATmega328P in Arduino Uno), Microchip PIC
- **16-bit:** MSP430 (ultra-low-power), dsPIC
- **32-bit:** ARM Cortex-M series (STM32, nRF52, LPC), ESP32 (Wi-Fi/BLE SoC)

1.3.3 Single Board Computers (SBCs)

SBCs like the Raspberry Pi, BeagleBone, and NVIDIA Jetson Nano integrate an application-class processor (ARM Cortex-A) with DRAM, display output, USB, Ethernet, and GPIO. They run full Linux and are used for rapid prototyping, IoT gateways, and edge computing.

1.3.4 Industrial Embedded Computers

Ruggedised embedded computers run deterministic real-time operating systems such as VxWorks, QNX, or INTEGRITY. They feature ECC memory, watchdog timers, and wide temperature ranges ($-40°C$ to $+85°C$). Applications include railway signalling, nuclear plant control, and avionics.

Table 1.3: Platform comparison: microprocessor vs. microcontroller vs. SBC vs. industrial PC.

Feature	**Microprocessor**	**Microcontroller**	**SBC**	**Industrial PC**
Integration	CPU only	CPU + peripherals	SoC + DRAM + I/O	Full system
Typical clock	100 MHz–2 GHz	1–600 MHz	1–2 GHz	1–3 GHz
RAM	External, MB–GB	On-chip, KB–MB	On-board, 1–8 GB	2–32 GB
OS	RTOS or bare-metal	Bare-metal or RTOS	Linux/Android	VxWorks/QNX/Linux
Unit cost	\$5–\$50	\$0.30–\$15	\$15–\$200	\$500–\$5000
Power	0.5–20 W	1 mW–1 W	2–15 W	10–100 W
Board complexity	High	Low	Medium	High
Typical use	Legacy, DSP	Sensors, control	Prototyping, IoT	Safety-critical

1.4 Design Challenges

Designing embedded systems presents a unique set of engineering challenges that distinguish the field from general software development.

1.4.1 Power Consumption

Battery-operated devices demand ultra-low-power design. Techniques include clock gating, dynamic voltage and frequency scaling (DVFS), sleep modes (from *idle* consuming $\sim$1 mA to *deep sleep* consuming $<1\,\mu$A), and duty-cycling peripherals.

1.4.2 Area and Cost

In high-volume products (millions of units), saving even \$0.10 per BOM translates to millions in savings. This drives the choice of the cheapest MCU that meets requirements and minimisation of external components.

1.4.3 Real-Time Response

Deterministic response to external events (interrupts) within microseconds is essential for control applications. This requires careful software design: short ISRs, bounded execution times, and predictable scheduling.

1.4.4 Reliability and Safety

Embedded systems in automotive, medical, and aerospace domains must achieve very high reliability. Techniques include watchdog timers, ECC on memory, redundant processing (TMR), FMEA analysis, and certification to domain-specific safety standards.

1.4.5 Security

Connected embedded devices (IoT) face threats including firmware extraction, side-channel attacks, buffer overflows, and man-in-the-middle attacks on communication. Secure boot, encrypted storage, TLS/DTLS communication, and hardware security modules (HSMs) are critical countermeasures.

1.4.6 Development Complexity

Embedded development involves cross-compilation, hardware debugging (JTAG/SWD), logic analysers, oscilloscopes, and constrained debugging environments. Unlike desktop software, bugs may manifest only in specific hardware-software interactions.

1.5 Case Study: Anti-Lock Braking System (ABS)

Example 1.1: Anti-Lock Braking System

An Anti-Lock Braking System (ABS) is a quintessential embedded system that illustrates all key embedded design principles.

Function: Prevent wheel lock-up during emergency braking by modulating brake pressure on each wheel independently.

Architecture:

- **Sensors:** Four wheel speed sensors (Hall effect or magnetoresistive) measuring rotation frequency.
- **Actuators:** Hydraulic solenoid valves (one per wheel) controlling brake fluid pressure.
- **Controller:** 32-bit MCU (e.g., Infineon AURIX TriCore) running ABS algorithm.
- **Communication:** CAN bus connecting ABS ECU with engine ECU and dashboard.

Constraints:

- **Hard real-time:** Brake pressure must be adjusted within 5 ms of detecting impending lock-up.
- **Safety:** ASIL D (highest automotive safety integrity level per ISO 26262).
- **Reliability:** Must operate correctly across −40°C to +125°C, vibration, and 15+ year lifespan.
- **Power:** Powered by vehicle 12 V battery; power is not a critical constraint.

Algorithm sketch: The ABS controller reads wheel speed every 1 ms, computes the

slip ratio λ for each wheel:

$$\lambda = \frac{v_{\text{vehicle}} - v_{\text{wheel}}}{v_{\text{vehicle}}}$$

If $\lambda > 0.15$ (threshold for impending lockup), the controller commands the solenoid valve to release brake pressure. When λ falls below 0.10, pressure is reapplied. This cycle repeats at 15–20 Hz, producing the characteristic ABS pulsation.

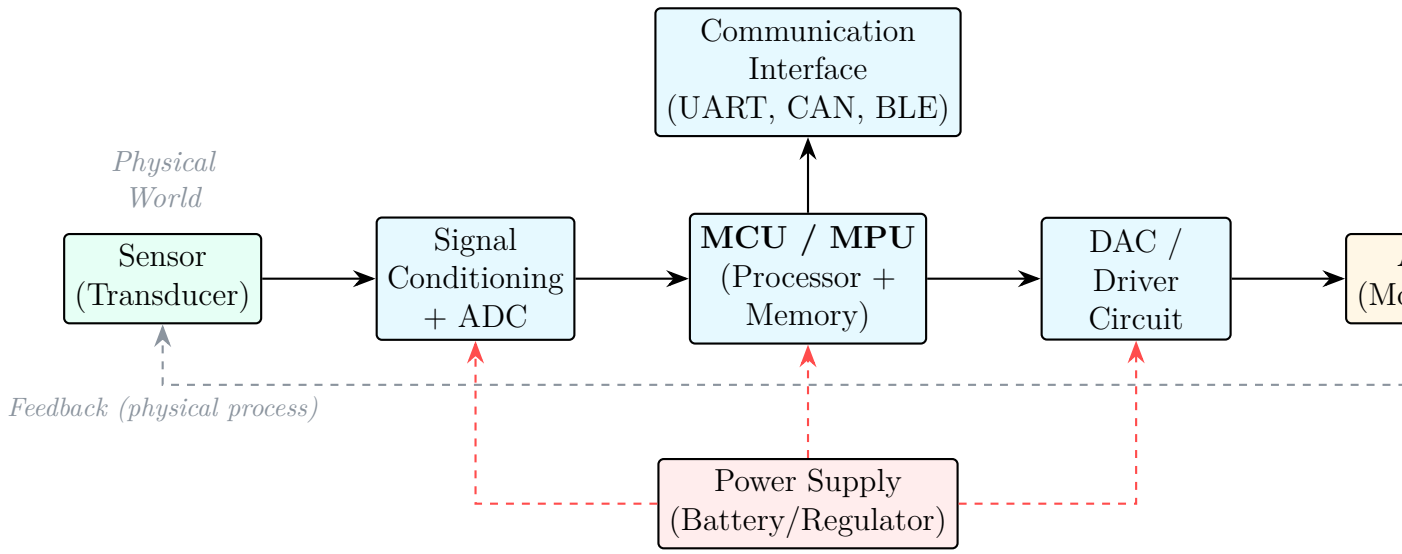

Figure 1.1: Generic embedded system block diagram: sensor → MCU → actuator with power supply and communication interface.

1.6 Worked Examples

Example 1.2: Classifying Embedded Systems

Classify each of the following systems as hard real-time, soft real-time, or non-real-time. Justify your answer.

(a) Cardiac pacemaker

(b) MP3 music player

(c) Digital thermometer for a kitchen oven

Solution

(a) **Hard real-time.** The pacemaker must deliver electrical stimulus to the heart within a precise timing window (typically ± 1 ms). Missing a deadline can cause cardiac arrhythmia or death.

(b) **Soft real-time.** Audio must be decoded and fed to the DAC at 44.1 kHz. Occasional missed samples cause audible glitches but not catastrophic failure.

(c) **Non-real-time.** The thermometer samples temperature every few seconds and displays it. A small delay in updating the display has no practical consequence.

Example 1.3: Estimating Memory Requirements

A sensor node samples a 12-bit ADC at 100 Hz and stores 10 minutes of data before transmitting. Estimate the minimum RAM required for the data buffer.

Solution

Each ADC sample is 12 bits, stored as a 16-bit (2-byte) integer.
Samples per 10 minutes:

$$N = 100 \times 60 \times 10 = 60{,}000 \text{ samples}$$

Memory required:

$$M = 60{,}000 \times 2 = 120{,}000 \text{ bytes} = 117.2 \text{ KB}$$

Therefore, the MCU needs at least 128 KB of RAM (nearest power-of-2), making a Cortex-M4 class MCU (e.g., STM32F4 with 192 KB SRAM) a suitable choice.

Example 1.4: Platform Selection

A company is designing a smart thermostat with a colour touchscreen, Wi-Fi connectivity, voice assistant integration, and a 5-year battery life goal. Which platform is most appropriate: 8-bit MCU, 32-bit MCU, SBC, or industrial PC? Justify.

Solution

A **32-bit MCU with integrated Wi-Fi** (e.g., ESP32-S3) is the most appropriate platform:

- A colour touchscreen requires a graphics driver with reasonable processing power—beyond 8-bit capability.
- Wi-Fi is required; the ESP32-S3 integrates Wi-Fi/BLE natively, avoiding external modules.
- Voice assistant integration can be offloaded to the cloud (the MCU handles only audio capture and streaming).
- A 5-year battery life is feasible with aggressive sleep modes ($<10\,\mu$A deep sleep on ESP32).
- An SBC (Raspberry Pi) would consume too much power ($\sim$2–5 W idle) for battery operation.
- An industrial PC is vastly over-specified in cost and power.

1.7 Practice Problems

Exercise 1.1: Embedded System Identification

Identify three embedded systems in a modern automobile other than ABS. For each, state the sensor(s), actuator(s), and the type of real-time constraint.

Exercise 1.2: Power Budget Estimation

A wearable fitness tracker uses a 100 mAh coin-cell battery. The MCU draws $500\,\mu$A in active mode and $2\,\mu$A in sleep mode. If the device is active for 5% of the time and sleeping for 95%, estimate the battery life in days.

Exercise 1.3: Platform Selection Exercise

A startup is building a drone flight controller that must process IMU data at 1 kHz, drive 4 ESCs via PWM, communicate via MAVLink over UART, and log data to an SD card. Recommend a platform class and justify your choice.

Exercise 1.4: Comparing Architectures

Draw a table comparing Von Neumann and Harvard memory architectures in terms of bus structure, instruction/data bandwidth, implementation complexity, and typical use cases. (This previews Module 2.)

Exercise 1.5: Design Challenge Analysis

For the following product, identify the top three design challenges and briefly explain how each would influence the choice of processor and OS: *"A portable ECG monitor that records heart signals for 48 hours and wirelessly uploads data to a hospital*

server."

Chapter 2

Microprocessor/Microcontroller Architecture

Module: 2 **Lecture Hours:** 5 **ILOs Addressed:** ILO-1, ILO-2, ILO-3

Learning Outcomes: By the end of this module, students will be able to:

1. Describe and compare Von Neumann and Harvard architectures.
2. Explain the 5-stage instruction pipeline and identify pipeline hazards.
3. Compare CISC and RISC instruction set philosophies.
4. Describe the internal architecture of a typical microcontroller including peripherals and interrupt handling.
5. Write and interpret basic ARM assembly instructions.

2.1 A Representative Microprocessor Architecture

A microprocessor is a digital integrated circuit that executes a stored program. Its core building blocks are the Arithmetic Logic Unit (ALU), the register file, the control unit, and the system buses.

2.1.1 Von Neumann vs. Harvard Architecture

Definition 2.1: Von Neumann Architecture

A computer architecture in which instructions and data share a single memory space and a single bus system. The processor fetches instructions and reads/writes data over the same bus, creating a potential bottleneck known as the *Von Neumann bottleneck*.

Definition 2.2: Harvard Architecture

A computer architecture with physically separate memory spaces and buses for instructions and data. This allows simultaneous instruction fetch and data access, increasing throughput.

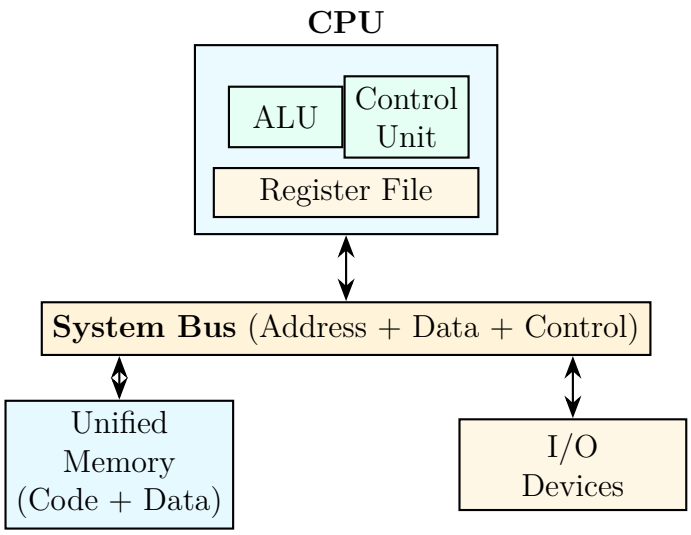

Figure 2.1: Von Neumann architecture: single shared bus for instructions and data.

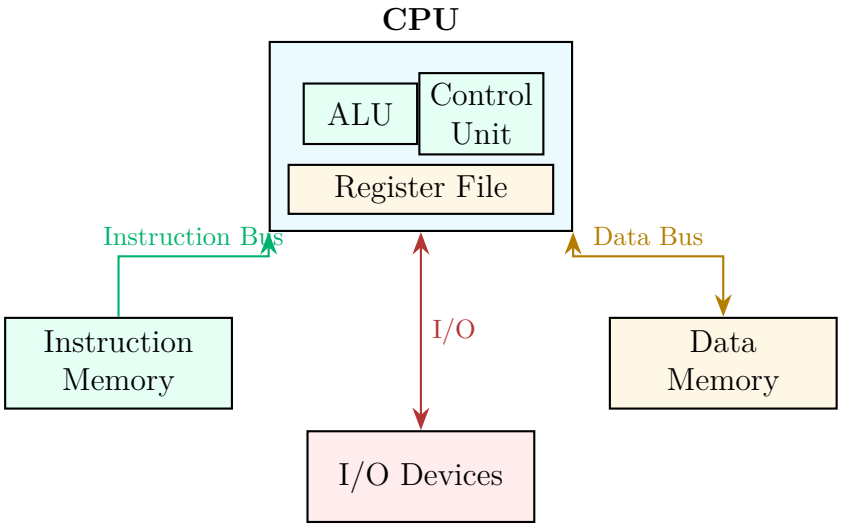

Figure 2.2: Harvard architecture: separate instruction and data buses enabling simultaneous access.

Table 2.1: Von Neumann vs. Harvard architecture comparison.

Attribute	Von Neumann	Harvard
Bus structure	Single shared bus	Separate instruction & data buses
Bandwidth	Limited (sequential fetch/data)	Higher (parallel access)
Complexity	Simpler wiring	More buses, more complex
Self-modifying code	Possible	Not straightforward
Typical use	Desktop CPUs, x86	DSPs, microcontrollers, ARM Cortex-M

◇ ***Insight***

Most modern processors use a modified Harvard architecture*: separate L1 instruction and data caches (Harvard) backed by a unified main memory (Von Neumann). ARM Cortex-M processors are a classic example — they use separate instruction and data buses (AHB-Lite) internally but present a unified memory map to the programmer.*

2.1.2 Core Components

- **Arithmetic Logic Unit (ALU):** Performs integer arithmetic (add, subtract, multiply) and bitwise logic (AND, OR, XOR, shift). The ALU receives operands from the register file and writes results back.
- **Register File:** A small, fast memory array (typically 16–32 registers, each 32 or 64 bits). Operands for ALU operations are loaded from registers; results are stored back. Special registers include the Program Counter (PC), Stack Pointer (SP), and Link Register (LR).
- **Control Unit (CU):** Decodes the current instruction and generates control signals that coordinate the ALU, register file, memory interface, and buses. In a hardwired CU, logic gates directly implement the control function. In a microprogrammed CU, a microinstruction ROM stores control sequences.

2.1.3 System Buses

Three buses interconnect the processor, memory, and peripherals:

1. **Address Bus:** Unidirectional; carries the memory address. Width determines addressable space: 2^n locations for an n-bit address bus.
2. **Data Bus:** Bidirectional; carries instruction opcodes and operand data. Width (8, 16, 32, 64 bits) determines how much data transfers per cycle.
3. **Control Bus:** Carries timing and control signals: read/write, interrupt request, bus grant, clock.

2.2 Instruction Set Architectures (ISA)

The ISA defines the programmer-visible interface to the processor: the instruction encoding, register model, addressing modes, and data types.

2.2.1 CISC vs. RISC Philosophy

Table 2.2: CISC vs. RISC design philosophies.

Attribute	CISC	RISC
Instruction count	Many (hundreds)	Few (dozens of base instructions)
Instruction complexity	Complex, multi-step	Simple, single-cycle ALU operations
Instruction length	Variable (1–15 bytes for x86)	Fixed (4 bytes for ARM, 2 bytes for Thumb)
Addressing modes	Many (memory-to-memory)	Few (load/store architecture)
Register count	Few (8 GPRs for x86)	Many (16–32 GPRs)
Pipeline friendliness	Hard to pipeline (variable decode)	Easy (uniform decode stage)
Code density	Higher (fewer instructions needed)	Lower (more instructions, but simpler)
Examples	x86, 68000, VAX	ARM, MIPS, RISC-V, PowerPC

△ Key Point: Load/Store Architecture

RISC processors follow the *load/store* paradigm: **only `LDR`** and **`STR`** instructions access memory. All arithmetic and logic instructions operate exclusively on registers. This simplifies the pipeline because memory access is confined to a single, predictable pipeline stage.

2.2.2 Instruction Formats

Using a MIPS-like encoding as an instructional example (similar principles apply to ARM):

- **R-type (Register):** Three register operands. Format: `opcode | rs | rt | rd | shamt | funct`. Example: **`ADD R3, R1, R2`**.
- **I-type (Immediate):** One source register + immediate constant. Format: `opcode | rs | rt | immediate`. Example: **`ADDI R3, R1, #5`**.
- **J-type (Jump):** Target address for unconditional jumps. Format: `opcode | address`. Example: **`J label`**.

2.2.3 Addressing Modes

Table 2.3: Common addressing modes in embedded processors.

Mode	Syntax Example	Effective Address
Immediate	`MOV R0, #42`	Operand is in the instruction itself
Register	`MOV R0, R1`	Operand is in register R1
Direct	`LDR R0, [0x2000]`	EA = address literal in instruction
Register indirect	`LDR R0, [R1]`	EA = contents of R1
Indexed (base+offset)	`LDR R0, [R1, #8]`	EA = R1 + 8
Pre-indexed with writeback	`LDR R0, [R1, #4]!`	EA = R1 + 4; R1 ← R1 + 4
Post-indexed	`LDR R0, [R1], #4`	EA = R1; then R1 ← R1 + 4
PC-relative	`LDR R0, [PC, #offset]`	EA = PC + offset (for constants)

2.2.4 ARM ISA Overview

The ARM architecture is the dominant ISA in embedded systems:

- **ARM state:** 32-bit fixed-length instructions, full instruction set.
- **Thumb state:** 16-bit compressed instructions for improved code density (at slight performance cost).
- **Thumb-2:** Mixed 16/32-bit instruction encoding used by all Cortex-M processors. Achieves near-ARM performance with Thumb-like code density.
- **Condition codes:** ARM instructions can be conditionally executed based on the APSR flags (N, Z, C, V), reducing branch overhead. Example: **`ADDEQ R0, R1, R2`** executes only if the Zero flag is set.

2.3 Instruction Fetch and Execution Cycles

2.3.1 Single-Cycle and Multi-Cycle Execution

In the simplest implementation, each instruction takes one clock cycle but the cycle must be long enough for the slowest instruction. Multi-cycle implementations break execution into stages, each taking one shorter clock cycle:

1. **Instruction Fetch (IF):** Read instruction from memory at address PC; increment PC.
2. **Instruction Decode (ID):** Decode opcode, read source registers from register file.

3. **Execute (EX):** ALU computes result (arithmetic) or effective address (load/store).
4. **Memory Access (MEM):** Read from or write to data memory (only for load/store instructions).
5. **Write Back (WB):** Write the result to the destination register.

2.3.2 Pipelining

Pipelining overlaps the execution of multiple instructions. While instruction i is in the EX stage, instruction $i+1$ is in ID, and instruction $i+2$ is in IF. Ideally, one instruction completes per cycle (CPI ≈ 1) after the pipeline fills.

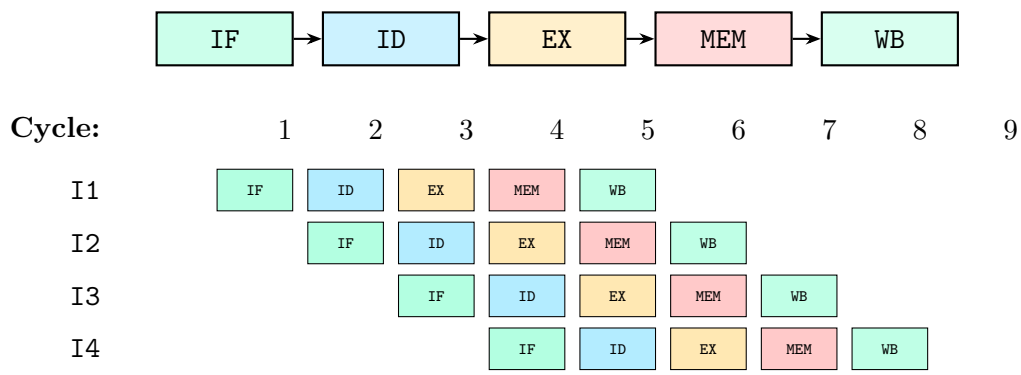

Figure 2.3: 5-stage instruction pipeline showing overlapped execution of four instructions.

2.3.3 Pipeline Hazards

Pipelining introduces three types of hazards:

Definition 2.3: Data Hazard

Occurs when an instruction depends on the result of a preceding instruction that has not yet completed its write-back stage. Example: `ADD R1, R2, R3` followed by `SUB R4, R1, R5` — the SUB reads R1 before ADD writes it.

Resolution techniques:

- **Stalling (bubbling):** Insert NOP cycles until the data is available. Simple but wastes cycles.
- **Forwarding (bypassing):** Route the ALU result directly from EX/MEM stage to the next instruction's EX input, bypassing the register file write. Eliminates most stalls.

Definition 2.4: Control Hazard

Occurs at branch instructions: the pipeline has already fetched subsequent instructions before knowing whether the branch is taken. A taken branch means the fetched

instructions are wrong.

Resolution:

- **Branch prediction:** Predict taken or not-taken; flush pipeline on misprediction.
- **Delayed branch:** Fill the branch delay slot with a useful instruction (common in MIPS).
- **Branch target buffer (BTB):** Cache recent branch targets for quick redirection.

Definition 2.5: Structural Hazard

Occurs when two pipeline stages need the same hardware resource simultaneously (e.g., a single-port memory for both IF and MEM stages).

Resolution: Duplicate resources (separate instruction and data caches — Harvard approach).

2.3.4 Branch Prediction

Modern processors use branch predictors to guess the outcome of conditional branches:

- **Static prediction:** Always predict not-taken (or always taken for backward branches in loops).
- **Dynamic prediction:** Use a 2-bit saturating counter per branch address. States: Strongly Not Taken → Weakly Not Taken → Weakly Taken → Strongly Taken. Achieves 85–95% accuracy.

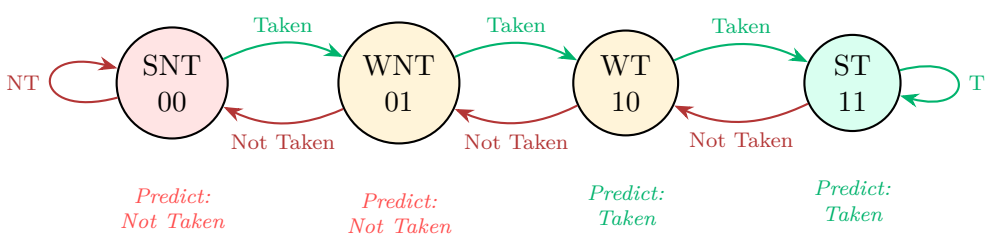

Figure 2.4: 2-bit saturating counter branch predictor state machine.

ARM Cortex-M3/M4: 3-Stage Pipeline

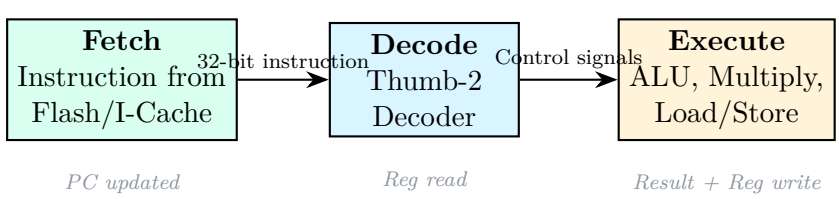

Figure 2.5: ARM Cortex-M3/M4 3-stage pipeline architecture.

▶ Practical Note

The ARM Cortex-M3/M4 uses a simplified **3-stage pipeline** (Fetch, Decode, Execute), not the classic 5-stage design. The Cortex-M7 extends this to a **6-stage pipeline** with branch prediction and dual-issue capability. The 3-stage design keeps interrupt latency low (12 cycles on Cortex-M3) at the cost of some throughput.

2.4 Microcontroller Internals

A microcontroller integrates the processor core with on-chip peripherals, eliminating the need for external components in many applications.

2.4.1 On-Chip Peripherals

Typical MCU peripherals include:

- **Timers:** General-purpose timers (counting, PWM generation, input capture), SysTick timer (ARM standard), real-time clock (RTC).
- **UART:** Asynchronous serial communication (RS-232 compatible logic levels).
- **SPI:** High-speed synchronous serial interface (full-duplex, up to 50 Mbps).
- **I^2C:** Two-wire synchronous serial bus (multi-master, 100/400/1000 kbps).
- **ADC:** Analog-to-digital converter (typically 12-bit SAR, multiple channels via multiplexer).
- **DAC:** Digital-to-analog converter (12-bit, for waveform generation).
- **GPIO:** General-purpose digital I/O pins, configurable as input or output with pull-up/pull-down.
- **DMA:** Direct Memory Access controller for high-throughput data transfer without CPU intervention.
- **Watchdog Timer (WDT):** Resets the processor if software fails to periodically refresh it, providing a recovery mechanism from software hangs.

2.4.2 Memory Map of a Typical MCU

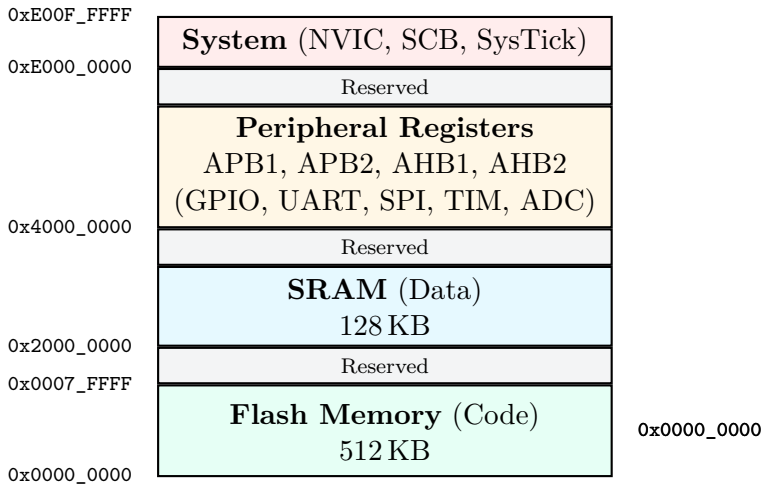

Figure 2.6: Simplified memory map of an STM32F4 microcontroller (Cortex-M4).

2.4.3 Interrupt Handling

Definition 2.6: Interrupt

An interrupt is a hardware or software signal that causes the processor to suspend its current execution, save context, and jump to an Interrupt Service Routine (ISR). Upon ISR completion, the processor restores context and resumes the interrupted program.

Key concepts in ARM Cortex-M interrupt handling:

- **Vector Table:** A table stored at the base of Flash memory (address `0x0000_0000`) containing the initial stack pointer value and addresses of all exception/interrupt handlers.
- **NVIC (Nested Vectored Interrupt Controller):** Hardware unit that manages interrupt priorities, enables/disables individual interrupts, and supports nested interrupts (a higher-priority interrupt can preempt a lower-priority ISR).
- **Exception Frame:** When an interrupt occurs, the hardware automatically pushes 8 registers onto the stack: R0–R3, R12, LR, PC, and xPSR. This takes 12 clock cycles on Cortex-M3.
- **Tail-chaining:** If a pending interrupt is waiting when an ISR completes, the NVIC skips the full context restore/save and directly enters the next ISR, saving 6 cycles.

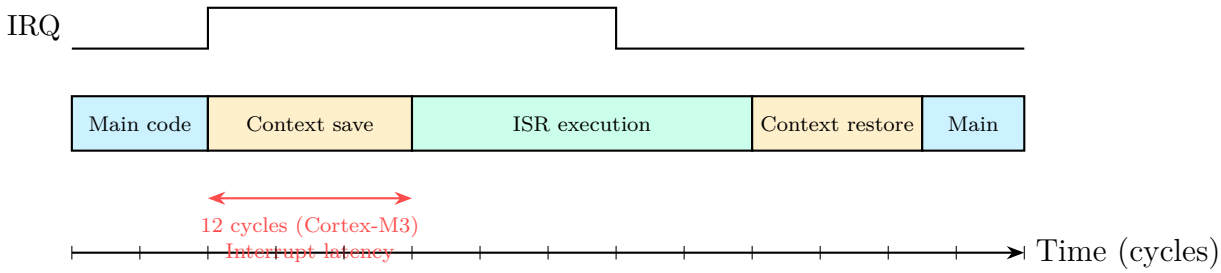

Figure 2.7: Interrupt latency: from IRQ assertion to first ISR instruction on ARM Cortex-M3.

2.5 ARM Assembly Code Examples

2.5.1 Data Processing Instructions

```
    ; R0 = a, R1 = b: compute R2 = 3*a + b - 1
    MOV  R2, R0          ; R2 = a
    ADD  R2, R2, R0, LSL #1 ; R2 = a + 2*a = 3*a
    ADD  R2, R2, R1      ; R2 = 3*a + b
    SUB  R2, R2, #1      ; R2 = 3*a + b - 1

    ; Bitwise operations
    AND  R3, R0, #0xFF   ; R3 = a & 0xFF (mask lower byte)
    ORR  R4, R0, R1      ; R4 = a | b
    EOR  R5, R0, R1      ; R5 = a ^ b (XOR)
```

Listing 2.1: ARM Cortex-M assembly: basic data processing.

2.5.2 Load/Store and Branch Instructions

```
    ; Load value from memory address stored in R0
    LDR  R1, [R0]        ; R1 = *R0 (word load)
    LDRB R2, [R0, #4]    ; R2 = *(uint8_t*)(R0 + 4)

    ; Store R3 to memory with post-increment
    STR  R3, [R0], #4    ; *R0 = R3; R0 += 4

    ; Conditional branch: if R1 == 0, jump to label
    CMP  R1, #0          ; Set flags: R1 - 0
    BEQ  zero_handler    ; Branch if Zero flag set

    ; Loop example: sum array of N words
    ; R0 = array base, R1 = N, R2 = sum (initially 0)
    MOV  R2, #0
loop:
```

```
    LDR  R3, [R0], #4   ; Load next element, advance pointer
    ADD  R2, R2, R3     ; Accumulate
    SUBS R1, R1, #1     ; Decrement counter, set flags
    BNE  loop           ; Branch if not zero
```

Listing 2.2: ARM Cortex-M assembly: memory access and branching.

2.5.3 C Code with Inline Assembly

```c
#include <stdint.h>

static inline uint32_t get_cycle_count(void) {
    uint32_t cycles;
    __asm volatile ("MRS %0, DWT_CYCCNT" : "=r" (cycles));
    return cycles;
    /* Note: DWT->CYCCNT must be enabled first:
       CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
       DWT->CYCCNT = 0;
       DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk; */
}

// Alternative using CMSIS register access:
uint32_t read_cycle_counter(void) {
    return DWT->CYCCNT;
}
```

Listing 2.3: C function using inline assembly to read the cycle counter (DWT) on Cortex-M.

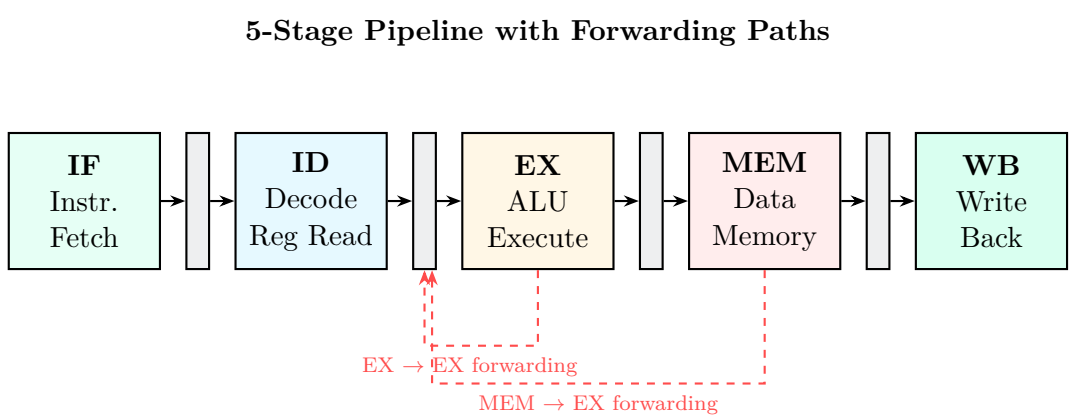

Figure 2.8: 5-stage pipeline datapath showing EX-to-EX and MEM-to-EX forwarding paths (dashed red arrows) that resolve data hazards without stalling.

2.6 Worked Examples

Example 2.1: Pipeline Timing Analysis

Consider the following instruction sequence on a 5-stage pipeline *without* forwarding:

```
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5   ; depends on R1 from I1
I3: AND R6, R1, R7   ; depends on R1 from I1
I4: ORR R8, R4, R6   ; depends on R4 from I2, R6 from I3
```

Determine the number of stall cycles and total execution cycles.

Solution

Without forwarding, a dependent instruction must wait until the producing instruction reaches WB:
I2 depends on `R1` from **I1**. I1 writes R1 in cycle 5 (WB). I2 needs R1 in cycle 3 (ID). $\Rightarrow$ 2-cycle stall.
I3 depends on `R1` from **I1**. After the 2-stall, I3 enters ID in cycle 6. I1 wrote R1 in cycle 5. $\Rightarrow$ No additional stall (data available).
I4 depends on `R4` from **I2** and `R6` from **I3**. I2 writes R4 in cycle 9. I4 enters ID at the earliest in cycle 8 $\Rightarrow$ 1-cycle stall for R4. I3 writes R6 in cycle 10. $\Rightarrow$ This is the later dependency; 2-cycle stall for R6.
Total stalls: $2 + 0 + 2 = 4$ stall cycles.
Without stalls: 4 instructions on 5-stage pipeline $= 5 + 3 = 8$ cycles. With stalls: $8 + 4 = 12$ cycles. Effective CPI: $12/4 = 3.0$.

Example 2.2: Pipeline with Forwarding

Repeat the previous example *with* EX-to-EX and MEM-to-EX forwarding.

Solution

With forwarding:
I2 depends on `R1` from **I1**: I1's ALU result is available at the end of EX stage (cycle 3). With EX-to-EX forwarding, I2 can read this value in its own EX stage (cycle 4). $\Rightarrow$ 0 stalls.
I3 depends on `R1` from **I1**: By the time I3 reaches EX (cycle 5), I1's result has already been written back. $\Rightarrow$ 0 stalls.
I4 depends on `R4` from **I2**: EX-to-EX forwarding provides R4. Also depends on `R6` from **I3**: EX-to-EX forwarding provides R6. $\Rightarrow$ 0 stalls.
Total stalls: 0. Total cycles: $5 + 3 = 8$. Effective CPI: $8/4 = 2.0$.

Example 2.3: Address Calculation

Given R1 = 0x2000_0100, what address does each instruction access?

```
(a) LDR R0, [R1]
(b) LDR R0, [R1, #8]
(c) LDR R0, [R1, #4]!
(d) LDR R0, [R1], #4
```

Solution

(a) **Register indirect:** EA = R1 = 0x2000_0100. R1 unchanged.

(b) **Base + offset:** EA = R1 + 8 = 0x2000_0108. R1 unchanged.

(c) **Pre-indexed with writeback:** EA = R1 + 4 = 0x2000_0104. After: R1 = 0x2000_0104.

(d) **Post-indexed:** EA = R1 = 0x2000_0100. After: R1 = R1 + 4 = 0x2000_0104.

Example 2.4: Interrupt Latency Calculation

An ARM Cortex-M4 running at 168 MHz receives an IRQ. The NVIC pushes 8 registers (12 cycles), and the ISR executes 50 instructions averaging 1.2 CPI. Calculate the total interrupt response time.

Solution

Clock period: $T_{clk} = 1/168\text{MHz} \approx 5.95\,\text{ns}$.
Context save: 12 cycles $\times 5.95\,\text{ns} = 71.4\,\text{ns}$.
ISR execution: $50 \times 1.2 = 60$ cycles $\times 5.95\,\text{ns} = 357\,\text{ns}$.
Context restore: 12 cycles $\times 5.95\,\text{ns} = 71.4\,\text{ns}$.
Total response time (from IRQ to ISR start): $71.4\,\text{ns} \approx 71\,\text{ns}$.
Total interrupt handling time (from IRQ to return to main): $71.4 + 357 + 71.4 \approx 500\,\text{ns}$.

Example 2.5: NVIC Priority Configuration

An STM32F4 uses 4 bits for interrupt priority (16 levels, 0 = highest). Configure priorities so that UART RX (IRQ 37) has higher priority than Timer 2 (IRQ 28), but both can preempt the SysTick handler.

Solution

Using CMSIS:

```
// Group priority = preemption, Sub-priority = tie-breaking
NVIC_SetPriorityGrouping(3); // 4 bits preemption, 0 sub-priority

NVIC_SetPriority(USART1_IRQn, 2); // UART: priority 2 (higher)
NVIC_SetPriority(TIM2_IRQn, 4); // Timer 2: priority 4
NVIC_SetPriority(SysTick_IRQn, 6); // SysTick: priority 6 (lowest)

NVIC_EnableIRQ(USART1_IRQn);
NVIC_EnableIRQ(TIM2_IRQn);
```

With these settings: UART (priority 2) can preempt Timer 2 (priority 4), and both can preempt SysTick (priority 6).

2.7 Practice Problems

Exercise 2.1: Architecture Comparison

Draw a detailed block diagram of a modified Harvard architecture as used in ARM Cortex-M processors. Label the I-bus, D-bus, S-bus, and show how they connect to Flash, SRAM, and peripherals via the bus matrix.

Exercise 2.2: Pipeline Hazard Detection

Identify all data hazards in the following sequence and indicate which can be resolved by forwarding:

```
ADD R1, R2, R3
SUB R4, R1, R5
LDR R6, [R4]
AND R7, R6, R1
```

Exercise 2.3: CPI Calculation

A program consists of 40% ALU instructions (1 cycle each), 30% load instructions (1 cycle + 20% cache miss rate causing 10-cycle penalty), 20% store instructions (1 cycle), and 10% branch instructions (1 cycle + 30% misprediction causing 3-cycle penalty). Calculate the average CPI.

Exercise 2.4: ARM Assembly Programming

Write an ARM Cortex-M assembly routine that computes the factorial of a number in `R0` (assume $0 \leq n \leq 12$) and returns the result in `R0`.

Exercise 2.5: Interrupt Priority

Three interrupts arrive simultaneously on an ARM Cortex-M3: External IRQ 0 (priority 3), UART1 RX (priority 1), and Timer 3 (priority 5). In what order are they serviced? What happens if Timer 3's ISR is executing and UART1 RX fires?

Exercise 2.6: Memory Bandwidth

A processor runs at 100 MHz with a 32-bit data bus. Calculate the peak memory bandwidth in MB/s. If the actual CPI is 1.5 and 35% of instructions are load/store, what is the effective memory bandwidth utilisation?

Exercise 2.7: Addressing Mode Selection

For each operation below, choose the most efficient ARM addressing mode and write the instruction:

(a) Load a constant 0xFF into R0.

(b) Access the 5th element of a word array whose base address is in R1.

(c) Walk through an array, loading each element and advancing the pointer.

(d) Access a peripheral register at a fixed address `0x4001_0000`.

Exercise 2.8: Branch Prediction Accuracy

A 2-bit predictor (initially "Strongly Not Taken") encounters the branch pattern T, T, NT, T, T, T, NT, T (T = taken, NT = not taken). Trace the predictor state for each branch and calculate the prediction accuracy.

Chapter 3

Embedded Processors

Module: 3 **Lecture Hours:** 4 **ILOs Addressed:** ILO-2, ILO-3, ILO-4

Learning Outcomes: By the end of this module, students will be able to:

1. Classify embedded processors by type and application domain.
2. Compare ARM Cortex-M, Cortex-R, and Cortex-A processor families.
3. Describe DSP architectures including the SHARC super-Harvard design.
4. Write bare-metal C code for register-level peripheral control.
5. Explain startup code, linker scripts, and cross-compilation toolchains.

3.1 Types of Embedded Processors

3.1.1 General-Purpose Microcontrollers

General-purpose MCUs are the workhorses of embedded design. They integrate a CPU core with on-chip Flash, SRAM, and peripherals for control-oriented tasks.

- **AVR (Atmel, now Microchip):** 8-bit RISC, Harvard architecture, widely used in Arduino. ATmega328P: 16 MHz, 32 KB Flash, 2 KB SRAM.
- **PIC (Microchip):** Available in 8/16/32-bit variants. PIC18F series popular in cost-sensitive applications.
- **STM32 (STMicroelectronics):** ARM Cortex-M based, spanning M0 (ultra-low-cost) to M7 (high-performance DSP). The STM32F4 series (168 MHz, Cortex-M4F) is widely used in university courses.
- **ESP32 (Espressif):** Dual-core Xtensa LX6, integrated Wi-Fi and Bluetooth, popular for IoT.

3.1.2 Digital Signal Processors (DSPs)

DSPs are optimized for repetitive, numerically intensive calculations found in signal processing:

- **Hardware MAC unit:** A Multiply-Accumulate unit that computes $a = a + b \times c$ in a single cycle, essential for FIR/IIR filters, FFT, and matrix operations.
- **Fixed-point arithmetic:** Many DSPs operate on Q15 or Q31 fixed-point formats, avoiding the area and power cost of floating-point hardware.
- **Floating-point DSPs:** Higher-end DSPs (TI TMS320C6748, Analog Devices SHARC) include IEEE-754 floating-point units.
- **VLIW (Very Long Instruction Word):** Multiple operations packed into a single wide instruction word, exploiting instruction-level parallelism. The compiler (not hardware) schedules parallel operations.
- **Circular buffers:** Hardware-supported modular addressing for implementing delay lines without software overhead.

3.1.3 Application Processors

Application processors (Cortex-A class) run feature-rich operating systems:

- ARM Cortex-A53/A72: 64-bit, out-of-order execution, MMU for virtual memory, multi-level caches.
- Multimedia extensions: NEON SIMD (Single Instruction, Multiple Data) for parallel 128-bit vector operations on 8/16/32-bit data lanes. Used for image/audio processing and machine learning inference.
- Used in smartphones, SBCs (Raspberry Pi), and infotainment systems.

3.1.4 FPGAs and Reconfigurable Computing

Definition 3.1: FPGA

A Field-Programmable Gate Array is an IC containing an array of configurable logic blocks (CLBs), interconnects, and I/O blocks that can be programmed after manufacturing to implement arbitrary digital circuits.

When to use FPGAs:

- Custom hardware acceleration (e.g., video codec, cryptography).
- Hard real-time requirements that software alone cannot meet.
- Low-to-medium production volumes where ASIC NRE cost is prohibitive.
- Rapid prototyping of digital designs before ASIC tape-out.

Vendors: Xilinx (now AMD) Artix/Kintex/Virtex, Intel (formerly Altera) Cyclone/Stratix, Lattice iCE40/ECP5.

3.1.5 ASICs (Application-Specific Integrated Circuits)

ASICs are custom-designed silicon for a single application. They offer the highest performance and lowest power per unit but require enormous NRE (Non-Recurring Engineering) costs ($1M–$100M+). Justified only at very high production volumes (millions of units) or where no other solution meets size/power targets (e.g., Bitcoin mining ASICs, smartphone SoCs).

3.2 ARM Processor Family

ARM Holdings licenses processor core designs (IP) to semiconductor companies who fabricate chips. The ARM Cortex family is organised into three profiles:

3.2.1 Cortex-M: Microcontroller Profile

- **Cortex-M0/M0+:** Smallest ARM core, 8–32 KB Flash, ideal for ultra-low-cost replacements of 8-bit MCUs. 2-stage pipeline, Thumb (16-bit) ISA subset.
- **Cortex-M3:** 3-stage pipeline, full Thumb-2 ISA, hardware divide, bit-banding. Typical: 72 MHz, 64–512 KB Flash.
- **Cortex-M4:** Cortex-M3 + DSP extensions (single-cycle MAC, SIMD) + optional single-precision FPU (M4F). Optimal for digital control and sensor fusion.
- **Cortex-M7:** 6-stage superscalar pipeline, branch prediction, dual-issue, I/D cache, double-precision FPU. Up to 480 MHz. Used in high-performance motor control and audio.

3.2.2 Cortex-R: Real-Time Profile

Designed for hard real-time applications requiring deterministic response:

- Tightly-coupled memories (TCM) for guaranteed single-cycle access.
- ECC on caches and memories for fault tolerance.
- Dual-core lockstep mode for safety-critical applications (detect CPU faults).
- Used in automotive (engine ECU), hard disk controllers, medical devices.

3.2.3 Cortex-A: Application Profile

Full-featured processors for running complex OS:

- Memory Management Unit (MMU) with virtual memory (page tables, TLB).
- Multi-level cache hierarchy (L1 I+D, L2, sometimes L3).
- NEON SIMD, TrustZone security extensions.
- Out-of-order execution, branch prediction, speculative execution.

Table 3.1: ARM Cortex family comparison matrix.

Feature	M0+	M4F	M7	R5F	A53
ISA	Thumb	Thumb-2	Thumb-2	Thumb-2	ARMv8-A
Pipeline	2-stage	3-stage	6-stage	8-stage	In-order 8-stage
MHz (typ.)	48	168	480	800	1500
FPU	None	SP	SP+DP	SP+DP	SP+DP+NEON
Cache	None	None	I+D, 64 KB	I+D, TCM	L1+L2, MB scale
MMU	No (MPU)	No (MPU)	No (MPU)	MPU	Full MMU
DSP	No	Yes	Yes	Yes	NEON SIMD
Area (μm^2)	12K gates	50K	120K	90K	400K+
Power (mW/MHz)	0.01	0.04	0.07	0.08	0.15
Use case	IoT sensor	Motor ctrl	Audio/HMI	Automotive	Smartphone

3.2.4 Thumb-2 ISA Deep Dive

All Cortex-M processors use the Thumb-2 instruction set encoding, which mixes 16-bit and 32-bit instructions:

- Most common ALU, branch, and load/store instructions fit in 16 bits (dense encoding).
- Complex operations (barrel shift + ALU, conditional execution via IT block, bit-field manipulation) use 32-bit encodings.
- The processor decodes instructions seamlessly; the programmer need not manually select between 16-bit and 32-bit.

◇ ***Insight***

Thumb-2 achieves code density comparable to x86 (CISC) while maintaining RISC execution efficiency. A typical Thumb-2 binary is 25–35% smaller than equivalent ARM (32-bit) code.

3.2.5 CMSIS Standard

The Cortex Microcontroller Software Interface Standard (CMSIS) provides:

- **CMSIS-Core:** Standard API for CPU registers, NVIC, SysTick (e.g., `NVIC_SetPriority()`).
- **CMSIS-DSP:** Optimised DSP library (FFT, FIR, matrix math) for Cortex-M4/M7.
- **CMSIS-RTOS:** Generic RTOS API abstracting FreeRTOS, RTX, etc.
- **CMSIS-Driver:** Standard peripheral driver interfaces.

3.3 SHARC Processors (Analog Devices)

Definition 3.2: SHARC

The Super Harvard Architecture Computer (SHARC) is a family of high-performance DSP processors by Analog Devices featuring a **dual-bus Harvard architecture** with separate program memory and two independent data memory buses (DM and PM data), enabling three simultaneous memory accesses per cycle.

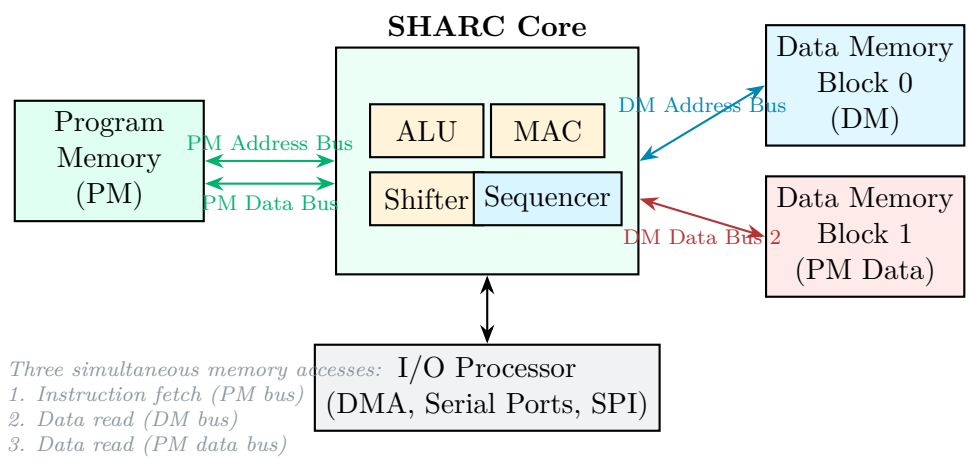

Figure 3.1: SHARC super-Harvard architecture with dual data memory buses enabling three parallel memory accesses.

Key SHARC features:

- **SIMD mode:** Two identical compute units process independent data streams in parallel.
- **VLIW execution:** A single 48-bit instruction word can specify a computation, two data moves, and a loop counter update simultaneously.
- **Hardware circular buffers:** DAG (Data Address Generator) units with modular arithmetic for zero-overhead delay lines.
- **Typical applications:** Professional audio processing (multi-channel mixing), software-defined radio, sonar/radar, real-time speech recognition.

3.4 Embedded Systems Programming

3.4.1 Bare-Metal C Programming Idioms

△ Key Point: The `volatile` Keyword

Any variable that can change outside the normal program flow (hardware registers, ISR-shared variables, DMA buffers) **must** be declared `volatile`. Without it, the compiler may optimize away reads/writes, causing the program to miss hardware state changes.

```c
#include <stdint.h>

// STM32F4 GPIO Port A base address
#define GPIOA_BASE  0x40020000UL

// Register offsets
#define GPIO_MODER  0x00  // Mode register
#define GPIO_ODR    0x14  // Output data register
#define GPIO_IDR    0x10  // Input data register
#define GPIO_BSRR   0x18  // Bit set/reset register

// Direct register access via pointer cast
#define REG(base, offset) \
    (*(volatile uint32_t *)((base) + (offset)))

void gpio_init_pa5_output(void) {
    // Set PA5 as general-purpose output (MODER bits [11:10] = 01)
    uint32_t moder = REG(GPIOA_BASE, GPIO_MODER);
    moder &= ~(0x3U << (5 *2)); // Clear bits 10:11
    moder |= (0x1U << (5 *2)); // Set bit 10 (output mode)
    REG(GPIOA_BASE, GPIO_MODER) = moder;
}

void gpio_toggle_pa5(void) {
    REG(GPIOA_BASE, GPIO_ODR) ^= (1U << 5);
}

uint32_t gpio_read_pa0(void) {
    return (REG(GPIOA_BASE, GPIO_IDR) >> 0) & 1U;
}
```

Listing 3.1: Memory-mapped I/O: direct register access for STM32 GPIO.

```c
typedef struct {
    volatile uint32_t MODER; // 0x00
    volatile uint32_t OTYPER; // 0x04
    volatile uint32_t OSPEEDR; // 0x08
```

```c
    volatile uint32_t PUPDR;  // 0x0C
    volatile uint32_t IDR;    // 0x10
    volatile uint32_t ODR;    // 0x14
    volatile uint32_t BSRR;   // 0x18
    volatile uint32_t LCKR;   // 0x1C
    volatile uint32_t AFR[2]; // 0x20-0x24
} GPIO_TypeDef;

#define GPIOA ((GPIO_TypeDef *) 0x40020000UL)
#define GPIOB ((GPIO_TypeDef *) 0x40020400UL)

// Usage: clean, readable, type-safe
void led_on(void) {
    GPIOA->BSRR = (1U << 5); // Set PA5 (atomic)
}

void led_off(void) {
    GPIOA->BSRR = (1U << 21); // Reset PA5 (atomic)
}
```

Listing 3.2: Struct overlay technique for register access.

3.4.2 Startup Code and Linker Scripts

When an ARM Cortex-M processor powers on or resets, it:

1. Reads the initial Stack Pointer value from address `0x0000_0000`.
2. Reads the Reset Handler address from address `0x0000_0004`.
3. Jumps to the Reset Handler (startup code).

The startup code (often called `crt0` or `startup_stm32.s`) performs:

```asm
    .syntax unified
    .cpu cortex-m4
    .thumb

    .section .isr_vector, "a"
    .word  _estack           ; Initial stack pointer
    .word  Reset_Handler     ; Reset vector
    .word  NMI_Handler       ; NMI
    .word  HardFault_Handler ; Hard fault
    ; ... (remaining vectors)

    .section .text
    .global Reset_Handler
    .type  Reset_Handler, %function
Reset_Handler:
```

```
    ; Copy .data section from Flash to SRAM
    ldr     r0, =_sdata         ; Start of .data in SRAM
    ldr     r1, =_edata         ; End of .data in SRAM
    ldr     r2, =_sidata        ; Start of .data init values in Flash
copy_data:
    cmp     r0, r1
    bge     zero_bss
    ldr     r3, [r2], #4
    str     r3, [r0], #4
    b       copy_data

    ; Zero-fill .bss section
zero_bss:
    ldr     r0, =_sbss
    ldr     r1, =_ebss
    mov     r2, #0
fill_bss:
    cmp     r0, r1
    bge     call_main
    str     r2, [r0], #4
    b       fill_bss

    ; Call main()
call_main:
    bl      SystemInit          ; Configure clocks (CMSIS)
    bl      main                ; Enter C application
    b       .                   ; Infinite loop if main returns
```

Listing 3.3: Minimal startup assembly for ARM Cortex-M (crt0-like).

```
/* Linker script for STM32F407 (512KB Flash, 128KB SRAM) */
MEMORY
{
  FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 512K
  SRAM  (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}

_estack = ORIGIN(SRAM) + LENGTH(SRAM); /* Top of stack */

SECTIONS
{
  .isr_vector : {
    KEEP(*(.isr_vector))
  } > FLASH

  .text : {
    *(.text*)         /* Code */
    *(.rodata*)       /* Read-only data (const) */
```

```
  } > FLASH

  _sidata = LOADADDR(.data);
  .data : {
    _sdata = .;
    *(.data*)         /* Initialized global/static variables */
    _edata = .;
  } > SRAM AT> FLASH /* VMA in SRAM, LMA in Flash */

  .bss : {
    _sbss = .;
    *(.bss*)
    *(COMMON)
    _ebss = .;
  } > SRAM

  /* Stack and heap */
  ._stack : {
    . = ALIGN(8);
    . = . + 4K;      /* 4KB stack */
  } > SRAM
}
```

Listing 3.4: Minimal linker script skeleton for ARM Cortex-M.

3.4.3 Memory Sections

Table 3.2: Standard memory sections in an embedded C program.

Section	Location	Contents
`.text`	Flash	Executable code (functions)
`.rodata`	Flash	Read-only constants, string literals
`.data`	Flash → SRAM	Initialized global/static variables (copied to SRAM at startup)
`.bss`	SRAM	Uninitialized global/static variables (zeroed at startup)
`.stack`	SRAM (top)	Runtime stack (grows downward on ARM)
`.heap`	SRAM	Dynamic memory (if used; generally discouraged in embedded)

3.4.4 Cross-Compilation Toolchain

Embedded software is developed on a *host* PC but runs on a *target* MCU. Key tools:

- **`arm-none-eabi-gcc`:** GCC cross-compiler for bare-metal ARM targets. Flags: `-mcpu=cortex-m4 -mthumb -mfloat-abi=hard -mfpu=fpv4-sp-d16`.

- **`arm-none-eabi-objdump`:** Disassemble ELF binaries to inspect generated assembly.
- **`arm-none-eabi-objcopy`:** Convert ELF to binary or Intel HEX format for flashing.
- **`arm-none-eabi-size`:** Report code size (`.text`), data (`.data`), and BSS (`.bss`).

▶ Practical Note

Always check the output of `arm-none-eabi-size` to verify your binary fits in the target's Flash and SRAM. Stack overflow is the most common cause of mysterious crashes in embedded systems.

3.5 Parallelism in Embedded Systems

3.5.1 Instruction-Level Parallelism (ILP)

- **Superscalar execution:** Multiple instructions issued per cycle (Cortex-M7 is dual-issue). Hardware detects independent instructions and executes them in parallel.
- **Out-of-order execution:** Instructions executed in data-dependency order, not program order (Cortex-A class). A reorder buffer (ROB) maintains in-order retirement.

3.5.2 Task-Level Parallelism

An RTOS schedules multiple tasks that run on the same CPU core, achieving *logical* parallelism through time-slicing. True parallelism requires multi-core processors (e.g., dual-core STM32H7, ESP32).

3.5.3 Hardware Parallelism

- **DMA controllers:** Transfer data between memory and peripherals without CPU involvement. The CPU configures the DMA transfer (source, destination, count) and is interrupted upon completion.
- **Peripheral co-processors:** Some MCUs include dedicated hardware for CRC calculation, AES encryption, or DSP operations that run independently of the main CPU.

3.5.4 SIMD for Embedded: ARM NEON

ARM NEON is a 128-bit SIMD extension available on Cortex-A and some DSP processors:

```c
#include <arm_neon.h>

void vector_add(float *a, float *b, float *c, int n) {
    for (int i = 0; i < n; i += 4) {
        float32x4_t va = vld1q_f32(&a[i]); // Load 4 floats
        float32x4_t vb = vld1q_f32(&b[i]);
```

```
        float32x4_t vc = vaddq_f32(va, vb); // SIMD add
        vst1q_f32(&c[i], vc);               // Store result
    }
}
```

Listing 3.5: NEON intrinsics example: vector addition of four 32-bit floats.

3.6 Worked Examples

Example 3.1: Processor Selection

An application requires 16-channel 12-bit ADC sampling at 1 MSPS total, a 256-tap FIR filter computed in real time, UART and SPI communication, and must cost under $5. Select an appropriate processor and justify.

Solution

The FIR filter requires 256 MAC operations per sample. At 1 MSPS, that is $256 \times 10^6 = 256$ MMAC/s.

An **ARM Cortex-M4F** (e.g., STM32F407 at 168 MHz) is ideal:

- Single-cycle MAC via DSP instructions: theoretical throughput = 168 MMAC/s. With overhead, approximately 100–120 MMAC/s sustained.
- For 256 MMAC/s, we need to reduce the filter to 128 taps, or increase clock to 480 MHz (Cortex-M7), or subsample.
- Actually, with 16 channels each at 62.5 kSPS: $256 \times 62{,}500 \times 16 = 256$ MMAC/s. This exceeds M4 capability.
- **Revised choice:** STM32H7 (Cortex-M7 at 480 MHz, dual-issue): ∼400 MMAC/s. Cost: ∼$4. Includes 16-channel ADC.

Example 3.2: Bitwise Register Manipulation

Configure STM32 GPIOB Pin 7 as output, push-pull, high-speed, no pull-up/pull-down, using direct register writes.

Solution

```c
// Enable GPIOB clock (RCC AHB1 peripheral clock enable register)
*(volatile uint32_t *)(0x40023830) |= (1U << 1); // RCC_AHB1ENR bit
    1

GPIO_TypeDef *GPIOB = (GPIO_TypeDef *)0x40020400UL;

// MODER: bits [15:14] = 01 (general purpose output)
GPIOB->MODER &= ~(3U << 14);
GPIOB->MODER |= (1U << 14);

// OTYPER: bit 7 = 0 (push-pull)
GPIOB->OTYPER &= ~(1U << 7);

// OSPEEDR: bits [15:14] = 10 (high speed)
GPIOB->OSPEEDR &= ~(3U << 14);
GPIOB->OSPEEDR |= (2U << 14);

// PUPDR: bits [15:14] = 00 (no pull-up/pull-down)
GPIOB->PUPDR &= ~(3U << 14);
```

Example 3.3: Memory Section Analysis

Given the following C code, identify which memory section each variable resides in:

```c
const char msg[] = "Hello";   // (a)
int counter = 42;             // (b)
static float buffer[256];     // (c)
void foo(void) {
    int local = 10;           // (d)
    static int call_count = 0; // (e)
    call_count++;
}
```

Solution

(a) `msg` is `const`: stored in **.rodata** (Flash).

(b) `counter` is an initialised global: stored in **.data** (initialized value in Flash, copied to SRAM at startup).

(c) `buffer` is an uninitialised static array: stored in **.bss** (SRAM, zeroed at startup).

(d) `local` is a local variable: stored on the **stack** (SRAM).

(e) `call_count` is a static local with an initialiser: stored in **.data** (like any initialized static).

Example 3.4: Cross-Compilation Size Analysis

After compiling, `arm-none-eabi-size` reports:

```
   text    data     bss     dec     hex filename
  12340     256    4096   16692    4134 firmware.elf
```

The target MCU has 64 KB Flash and 20 KB SRAM. Will the firmware fit?

Solution

Flash usage: `.text` + `.data` = 12,340 + 256 = 12,596 bytes = 12.3 KB. Flash has 64 KB. ✓
SRAM usage: `.data` + `.bss` = 256 + 4,096 = 4,352 bytes = 4.25 KB. Plus stack (typically 1–4 KB). Total ≈ 5–8 KB. SRAM has 20 KB. ✓
The firmware fits comfortably with room for growth.
Note: `.data` appears in *both* calculations because its initial values are stored in Flash (LMA) and copied to SRAM (VMA).

3.7 Practice Problems

Exercise 3.1: Processor Classification

Classify each processor below as MCU, DSP, Application Processor, FPGA, or ASIC: (a) STM32F103, (b) TMS320C6748, (c) Xilinx Artix-7, (d) Broadcom BCM2711 (Raspberry Pi 4), (e) Apple A16 Bionic, (f) ATmega328P, (g) ADSP-21489 SHARC.

Exercise 3.2: ARM Cortex Selection

For each application, recommend the most appropriate ARM Cortex profile (M, R, or A) and justify: (a) Smart thermostat with LCD display, (b) Automotive airbag controller, (c) Smartphone application processor, (d) Battery-powered BLE sensor

beacon.

Exercise 3.3: Register Manipulation

Write C code using the struct overlay technique to:

(a) Configure PA0 as input with internal pull-up resistor on STM32.

(b) Read the state of PA0 and return 1 if pressed (active low), 0 otherwise.

Exercise 3.4: Linker Script Interpretation

Given a linker script that places `.isr_vector` and `.text` in Flash starting at `0x0800_0000` and `.data` and `.bss` in SRAM starting at `0x2000_0000`: (a) What are the first 8 bytes at `0x0800_0000`? (b) What happens if `.data` LMA is not specified as Flash?

Exercise 3.5: SHARC vs. Cortex-M4 DSP

Compare the SHARC ADSP-21489 with the Cortex-M4F for implementing a 512-point complex FFT. Consider: MAC throughput, memory bandwidth, power consumption, and cost.

Exercise 3.6: Parallelism Analysis

A sensor fusion algorithm on Cortex-M4 reads accelerometer data via SPI (DMA), applies a Kalman filter (CPU), and transmits results via UART (DMA). Draw a timeline showing which hardware units are active during each phase and identify where parallelism is achieved.

Exercise 3.7: Code Size Optimization

Your firmware uses 63.5 KB of a 64 KB Flash. You need to add a new 2 KB function. List three techniques to reduce code size without changing functionality.

Chapter 4

Memory Architectures

Module: 4 **Lecture Hours:** 2 **ILOs Addressed:** ILO-2, ILO-3

Learning Outcomes: By the end of this module, students will be able to:

1. Describe SRAM, DRAM, Flash, and emerging memory technologies.
2. Explain the memory hierarchy and cache operation.
3. Interpret the memory map of an embedded processor.
4. Analyse cache performance using hit/miss rates and latency models.

4.1 Memory Technologies

4.1.1 SRAM (Static Random-Access Memory)

Definition 4.1: SRAM

Static RAM stores each bit in a bistable latch (flip-flop) constructed from six transistors (6T cell). Data is retained as long as power is supplied—no refresh is needed.

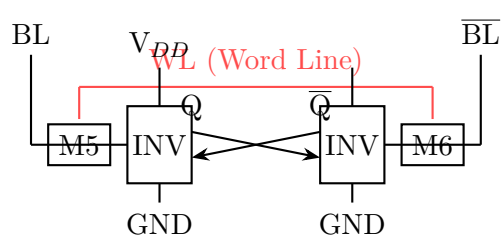

Figure 4.1: Simplified 6T SRAM cell: two cross-coupled inverters with two access transistors controlled by the word line.

Properties: fast access (1–10 ns), low density (6 transistors per bit), relatively high power (leakage currents), used for CPU registers, cache memory, and MCU on-chip SRAM.

4.1.2 DRAM (Dynamic Random-Access Memory)

DRAM stores each bit as a charge on a tiny capacitor (1T1C cell). The charge leaks over time, so DRAM requires periodic *refresh* (every 64 ms typically). DRAM is denser (1 transistor + 1 capacitor per bit) and cheaper per bit than SRAM but slower (50–100 ns latency). Used as main memory in application processors. Rarely used on-chip in MCUs due to refresh complexity.

4.1.3 Flash Memory

- **NOR Flash:** Byte-addressable reads (random access), supporting execute-in-place (XIP). Used for code storage in MCUs. Read: 10–100 ns, erase: 0.5–1 s per sector.
- **NAND Flash:** Page-oriented reads (512B–8KB pages), higher density and lower cost. Used for data storage (SD cards, eMMC, SSDs). Requires a Flash Translation Layer (FTL) for wear leveling.
- **Endurance:** NOR: 10,000–100,000 erase cycles. NAND: 1,000–100,000 cycles (depends on technology node).
- **Write mechanism:** Electrons are tunneled through oxide onto a floating gate (programming). Erasing removes electrons via Fowler-Nordheim tunneling for an entire block.

4.1.4 EEPROM

Electrically Erasable Programmable ROM: byte-addressable writes (unlike Flash which erases in blocks). Low endurance (100,000–1M cycles). Used for storing configuration parameters (e.g., calibration data, MAC addresses).

4.1.5 ROM Variants

- **Mask ROM:** Programmed at fabrication. Zero cost per-unit for large volumes, but inflexible.
- **OTP (One-Time Programmable):** Fuse-based, programmed once after fabrication.

4.1.6 Emerging Non-Volatile Memories

- **MRAM (Magnetoresistive RAM):** Uses magnetic tunnel junctions. Fast ($\sim$10 ns), high endurance ($> 10^{12}$ cycles), non-volatile. Increasingly used in safety-critical MCUs.
- **FeRAM (Ferroelectric RAM):** Based on ferroelectric capacitors. Low power, fast writes, but limited density. Used in TI MSP430FR series (FRAM-based MCU).

Table 4.1: Memory technology comparison.

Technology	Speed	Density	Power	Cost/bit	Volatile?	Endurance
SRAM	1–10 ns	Low	Medium	High	Yes	∞
DRAM	50–100 ns	High	Medium	Low	Yes	∞
NOR Flash	100 ns	Medium	Low	Medium	No	100K cycles
NAND Flash	25 μs	Very High	Low	Very Low	No	10K cycles
EEPROM	200 ns	Low	Low	High	No	1M cycles
MRAM	10 ns	Medium	Low	High	No	$> 10^{12}$
FeRAM	50 ns	Medium	V. Low	Medium	No	$> 10^{12}$

4.2 Memory Hierarchy

The memory hierarchy exploits the principle that *faster memory is more expensive*, creating a pyramid-shaped structure.

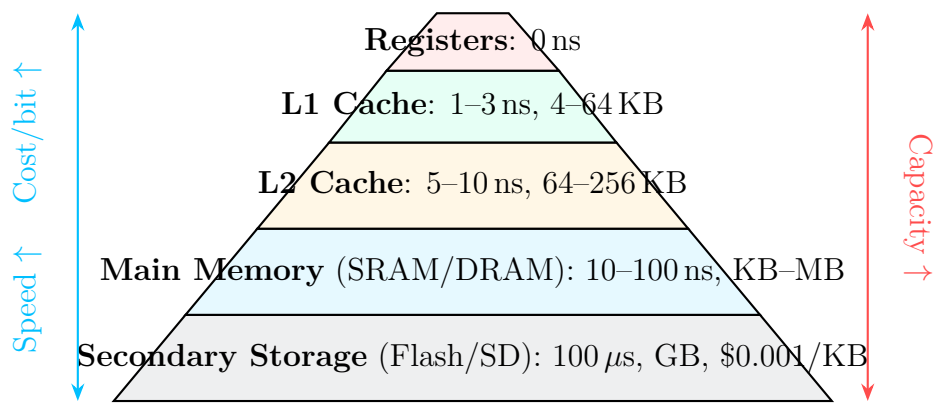

Figure 4.2: Memory hierarchy pyramid: speed and cost increase towards the top; capacity increases towards the bottom.

4.2.1 Locality of Reference

The memory hierarchy works because programs exhibit *locality*:

- **Temporal locality:** Recently accessed data is likely to be accessed again soon (e.g., loop variables).
- **Spatial locality:** Data near recently accessed data is likely to be accessed soon (e.g., array traversal).

4.2.2 Cache Operation

A cache is a small, fast memory that stores copies of frequently accessed main memory locations.

Definition 4.2: Cache Line

The fundamental unit of data transfer between cache and main memory. A typical cache line is 32 or 64 bytes. When a cache miss occurs, the entire cache line containing the requested address is fetched.

Cache associativity determines where a given memory block can be placed in the cache:

- **Direct-mapped:** Each memory block maps to exactly one cache line (determined by address bits). Fast lookup but susceptible to conflict misses.
- **N-way set-associative:** Each block can map to any of N lines within a set. The cache is divided into sets of N ways. Reduces conflict misses at the cost of more comparators.
- **Fully associative:** A block can be placed in any cache line. Eliminates conflict misses but requires searching all lines (expensive for large caches).

△ Key Point: Cache Performance

Average memory access time:

$$T_{\text{avg}} = T_{\text{hit}} + (1 - h) \times T_{\text{miss penalty}}$$

where h is the hit rate and T_{hit} is the cache access latency (typically 1 cycle for L1).

4.2.3 Cache Coherency in Multicore Systems

In multicore embedded systems (e.g., dual-core STM32H7), if both cores cache the same data, a write by one core may leave the other core's cache stale. Solutions:

- **Write-through:** Writes go to both cache and main memory. Simple but high bus traffic.
- **Write-back with snooping:** Write only to cache; snoop bus traffic to detect when another core writes to a shared line.
- **Software cache management:** Explicitly invalidate or clean cache lines before sharing data (common in Cortex-M7 with DMA).

4.3 Memory Models and Mapping

4.3.1 Physical vs. Virtual Memory

- **Physical address space:** The actual hardware addresses. What appears on the address bus. Cortex-M processors use only physical addresses (no MMU).

- **Virtual address space:** An abstraction provided by the MMU (on Cortex-A class). Programs use virtual addresses; the MMU translates them to physical addresses via page tables. Enables process isolation, demand paging, and memory protection.

4.3.2 Memory-Mapped I/O

In ARM (and most embedded architectures), peripheral registers are accessed at specific addresses in the memory map, just like SRAM. This means the same `LDR`/`STR` instructions access both memory and peripherals.

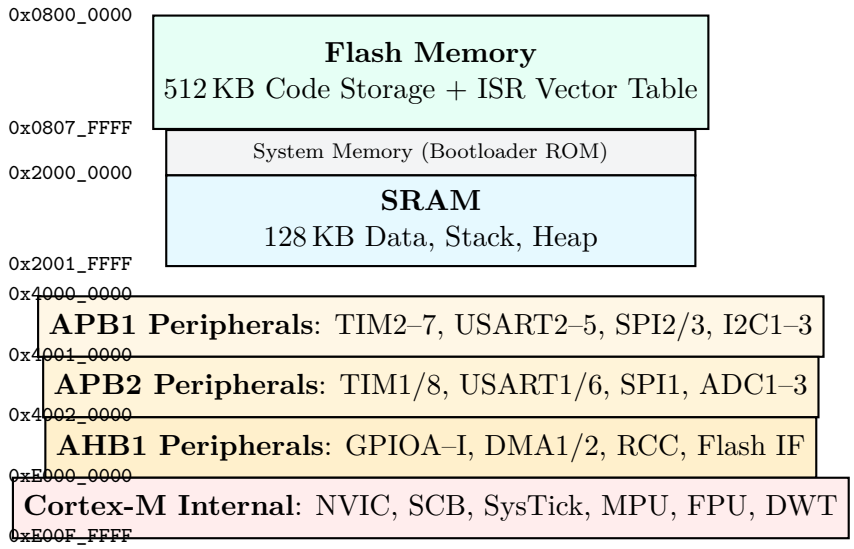

Figure 4.3: STM32F407 memory map showing Flash, SRAM, peripheral, and Cortex-M system address regions.

4.3.3 DMA (Direct Memory Access)

DMA controllers transfer data between memory regions or between peripherals and memory without CPU intervention. The CPU programs the DMA with source address, destination address, transfer count, and transfer width, then is free to perform other work. An interrupt signals DMA completion.

Transfer types:

- **Memory-to-memory:** Fast block copy (e.g., framebuffer update).
- **Peripheral-to-memory:** ADC samples → buffer, UART receive → buffer.
- **Memory-to-peripheral:** Buffer → DAC, buffer → SPI transmit.

▶ Practical Note: DMA and Cache Coherency

On Cortex-M7 with D-cache enabled, DMA writes to SRAM are not visible to the CPU until the cache is invalidated. Before reading DMA-written data, call `SCB_InvalidateDCache_by_Addr()`. Before DMA reads from SRAM the CPU just wrote, call `SCB_CleanDCache_by_Addr()`.

```c
// Method 1: Direct pointer (verbose, error-prone)
#define RCC_AHB1ENR (*(volatile uint32_t *)0x40023830UL)
RCC_AHB1ENR |= (1U << 0); // Enable GPIOA clock

// Method 2: Struct overlay (clean, maintainable)
typedef struct {
    volatile uint32_t CR;        // 0x00
    volatile uint32_t PLLCFGR;   // 0x04
    volatile uint32_t CFGR;      // 0x08
    volatile uint32_t CIR;       // 0x0C
    /* ... more registers ... */
    volatile uint32_t AHB1ENR;   // 0x30
} RCC_TypeDef;

#define RCC ((RCC_TypeDef *)0x40023800UL)

// Enable GPIOA and GPIOB clocks
RCC->AHB1ENR |= (1U << 0) | (1U << 1);
```

Listing 4.1: Accessing memory-mapped registers: direct pointer vs. struct overlay.

4.4 Worked Examples

Example 4.1: Cache Hit Rate Calculation

A Cortex-M7 with a 16 KB direct-mapped L1 data cache (32-byte lines) executes a program that sequentially reads a 64 KB array of 32-bit integers. What is the L1 hit rate assuming the cache is initially empty?

Solution

Cache parameters:

- Cache size = 16 KB = 16,384 bytes
- Line size = 32 bytes ⇒ 8 words (32-bit) per line
- Number of cache lines = 16,384/32 = 512

Array access: 64 KB / 4 bytes = 16,384 reads, sequential.

For each cache line fetched, the first access (1 word) is a **compulsory miss**, and the next 7 words hit in the cache (spatial locality).

After reading 16 KB (= cache size), the 17th KB starts evicting the 1st KB's cache lines (direct-mapped conflict). However, since we access sequentially and never revisit, there are no conflict misses—only compulsory misses.

Compulsory misses = total lines accessed = 64,000/32 = 2,048. Total accesses = 16,384.

Hit rate:

$$h = 1 - \frac{2{,}048}{16{,}384} = 1 - \frac{1}{8} = \frac{7}{8} = 87.5\%$$

This matches the expected result: each 32-byte line serves 8 word accesses, so 7 out of 8 are hits.

Example 4.2: Average Memory Access Time

A system has L1 cache (1 cycle, 95% hit rate), L2 cache (10 cycles, 80% hit rate for L1 misses), and main memory (100 cycles). Calculate the average memory access time (AMAT).

Solution

$$\begin{aligned}\text{AMAT} &= T_{L1} + (1 - h_{L1}) \times [T_{L2} + (1 - h_{L2}) \times T_{\text{mem}}] \\ &= 1 + 0.05 \times [10 + 0.20 \times 100] \\ &= 1 + 0.05 \times [10 + 20] \\ &= 1 + 0.05 \times 30 = 1 + 1.5 = 2.5 \text{ cycles}\end{aligned}$$

The memory hierarchy reduces effective access time from 100 cycles to 2.5 cycles — a 40× improvement.

Example 4.3: Memory Section Sizing

An embedded project has the following memory usage:

- Application code: 48 KB
- Read-only constants: 8 KB
- Initialized variables: 2 KB
- Uninitialized buffers: 12 KB

- Stack requirement: 4 KB

Determine the Flash and SRAM requirements.

Solution

Flash stores: `.text` (code) + `.rodata` (constants) + `.data` initialization values.

$$\text{Flash} = 48 + 8 + 2 = 58\,\text{KB}$$

SRAM stores: `.data` (runtime copy) + `.bss` (zeroed) + stack.

$$\text{SRAM} = 2 + 12 + 4 = 18\,\text{KB}$$

A suitable MCU: STM32F103 (64 KB Flash, 20 KB SRAM) or STM32F401 (256 KB Flash, 64 KB SRAM) for headroom.

4.5 Practice Problems

Exercise 4.1: Technology Selection

A wearable medical device needs 256 KB of non-volatile storage that retains calibration data through power cycles, with low write power and at least 10^{10} write endurance. Which memory technology is most appropriate: Flash, EEPROM, MRAM, or FeRAM? Justify.

Exercise 4.2: Cache Associativity Analysis

A 4-way set-associative cache has 256 lines total, 64-byte line size, and uses LRU replacement. How many sets are there? For a 32-bit address, show the tag, set index, and byte offset bit fields.

Exercise 4.3: Cache Miss Classification

For a direct-mapped cache with 8 lines (line size 16 bytes), trace the following byte address access sequence and classify each access as compulsory miss, conflict miss, or hit: 0, 16, 128, 0, 256, 0, 16, 128.

Exercise 4.4: DMA Configuration

The ADC on an STM32 produces 12-bit samples at 1 MSPS. Configure a DMA to transfer samples into a circular buffer of 1000 entries in SRAM. What is the required DMA bandwidth in bytes/second?

Exercise 4.5: Memory Map Interpretation

Given the STM32F407 memory map, explain why writing to address `0x0800_0000` does not directly modify Flash content (even though this is the Flash address range).

What procedure is required to write to Flash?

Chapter 5

I/O Ports

Module: 5 **Lecture Hours:** 2 **ILOs Addressed:** ILO-3, ILO-4

Learning Outcomes: By the end of this module, students will be able to:

1. Configure GPIO pins for digital input/output with pull-up/pull-down resistors.
2. Compare polling, interrupt-driven, and DMA-based I/O strategies.
3. Describe ADC and DAC operation, including resolution, sampling, and reconstruction.
4. Draw and interpret timing diagrams for UART, SPI, and I^2C protocols.
5. Write driver code for serial communication peripherals.

5.1 I/O Hardware

5.1.1 Digital GPIO

General-Purpose I/O pins can be configured as:

- **Input mode:** High-impedance state reads external voltage. Options:
  - **Floating:** No internal resistor; susceptible to noise if unconnected.
  - **Pull-up:** Internal resistor to V_{DD} ($\sim$40 kΩ); pin reads HIGH when undriven.
  - **Pull-down:** Internal resistor to GND; pin reads LOW when undriven.
- **Output mode:**
  - **Push-pull:** Actively drives both HIGH and LOW. Most common.
  - **Open-drain:** Drives LOW or floats (requires external pull-up for HIGH). Used for I^2C, level shifting, wired-OR bus sharing.
- **Alternate function:** Pin connects to an on-chip peripheral (UART TX, SPI CLK, etc.) instead of the GPIO data register.
- **Analog mode:** Pin connects directly to ADC/DAC input; digital input buffer disabled to reduce power.

5.1.2 Analog I/O

- **ADC (Analog-to-Digital Converter):** Converts a continuous analog voltage to a discrete digital value. Common types:
  - **SAR (Successive Approximation Register):** Most common in MCUs. 12-bit resolution, 1–5 MSPS.
  - **Sigma-Delta ($\Sigma\Delta$):** Very high resolution (16–24 bits) but slower. Used for precision measurement.
- **DAC (Digital-to-Analog Converter):** Converts a digital value to an analog voltage. Typically 12-bit on STM32. Used for waveform generation, audio output, and control voltage.

5.1.3 Signal Conditioning

Between the physical sensor and the MCU:

- **Op-amp buffer:** Impedance matching, preventing the sensor from being loaded by the ADC.
- **Level shifting:** Converting 5 V sensor output to 3.3 V MCU input (resistor divider or dedicated level shifter IC).
- **Anti-aliasing filter:** Low-pass filter before ADC to attenuate frequencies above $f_s/2$ (Nyquist).
- **ESD protection:** TVS diodes or clamping diodes on external-facing pins.

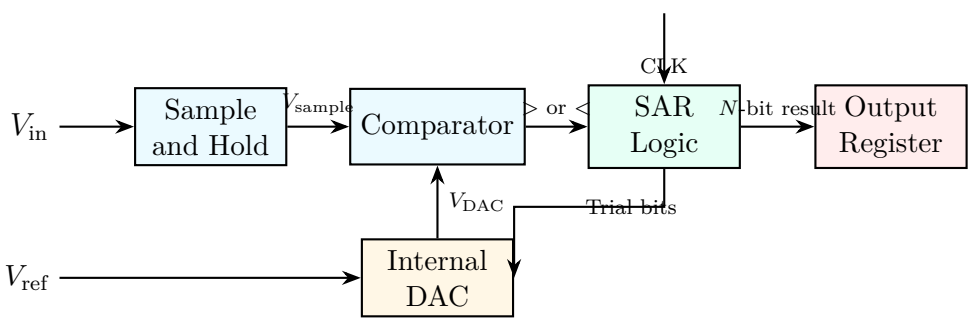

Figure 5.1: SAR ADC block diagram: successive approximation register with internal DAC feedback.

5.2 Microprocessor/Microcontroller Interfacing

5.2.1 Polling vs. Interrupt-Driven I/O vs. DMA

Table 5.1: I/O strategies comparison.

Aspect	Polling	Interrupt	DMA
CPU involvement	Continuous checking	On event only	Setup only
Latency	Variable (depends on poll rate)	Deterministic (ISR latency)	Near-zero CPU latency
CPU utilisation	Wastes cycles waiting	Efficient	Most efficient
Complexity	Simple	Moderate (ISR design)	Complex (coherency)
Throughput	Low	Medium	Highest
Use case	Simple, low-rate I/O	Most peripheral I/O	High-speed data transfer

5.2.2 ISR Design Rules

△ Key Point: Golden Rules for Interrupt Service Routines

1. **Keep ISRs short:** Do minimal work; defer heavy processing to main loop or a task.
2. **No blocking calls:** Never call `delay()`, `printf()`, or `malloc()` in an ISR.
3. **Use volatile for shared data:** Any variable written in an ISR and read in main code must be `volatile`.
4. **Clear the interrupt flag:** Most peripherals require software to clear the pending bit, or the ISR will re-trigger infinitely.
5. **Avoid priority inversion:** Assign priorities thoughtfully; higher-rate interrupts get higher priority.

5.2.3 GPIO Register Programming

AVR style (ATmega328P):

- `DDRx`: Data Direction Register (0 = input, 1 = output).
- `PORTx`: Output data / pull-up enable.
- `PINx`: Input data (read pin state).

STM32 style (Cortex-M):

- `GPIOx_MODER`: Mode register (00 = input, 01 = output, 10 = AF, 11 = analog), 2 bits per pin.
- `GPIOx_ODR`: Output Data Register.
- `GPIOx_IDR`: Input Data Register.
- `GPIOx_BSRR`: Bit Set/Reset Register (atomic set/reset).

5.3 Analog and Digital Interfaces

5.3.1 ADC: Resolution and Sampling

For an N-bit ADC with reference voltage V_{ref}:

$$\text{LSB} = \frac{V_{\text{ref}}}{2^N}, \quad D = \left\lfloor \frac{V_{\text{in}} \times 2^N}{V_{\text{ref}}} \right\rfloor$$

- **ENOB (Effective Number of Bits):** Accounts for noise and non-linearity. Typically 0.5–2 bits less than nominal resolution.
- **SNR:** $\text{SNR} = 6.02 \times \text{ENOB} + 1.76\,\text{dB}$.
- **Sampling Theorem:** $f_s \geq 2f_{\max}$ (Nyquist rate). In practice, oversample by 4–10×.
- **Anti-aliasing filter:** Must attenuate signal components above $f_s/2$ before the ADC samples.

5.3.2 PWM (Pulse Width Modulation)

PWM generates a digital waveform with variable duty cycle:

$$V_{\text{avg}} = V_{DD} \times \frac{t_{\text{on}}}{T} = V_{DD} \times D$$

where D is the duty cycle (0–100%). Used for motor speed control, LED dimming, and as a poor-man's DAC (with low-pass filter).

Timer-based PWM: A hardware timer counts up to a period value (ARR). The output pin goes HIGH when counter < compare value (CCR), and LOW otherwise.

```c
uint16_t adc_read_channel(uint8_t channel) {
    ADC1->SQR3 = channel;         // Select channel
    ADC1->CR2 |= ADC_CR2_SWSTART; // Start conversion

    while (!(ADC1->SR & ADC_SR_EOC)) // Poll for end-of-conversion
        ;

    return (uint16_t)ADC1->DR;    // Read 12-bit result
}
```

Listing 5.1: ADC read with polling on STM32.

```c
volatile uint16_t adc_result;
volatile uint8_t adc_ready = 0;

void ADC_IRQHandler(void) {
    if (ADC1->SR & ADC_SR_EOC) {
        adc_result = (uint16_t)ADC1->DR; // Read clears EOC flag
        adc_ready = 1;
```

```
    }
}

void adc_start_conversion(uint8_t channel) {
    ADC1->SQR3 = channel;
    ADC1->CR1 |= ADC_CR1_EOCIE;  // Enable end-of-conversion interrupt
    ADC1->CR2 |= ADC_CR2_SWSTART;
}
```

Listing 5.2: ADC read with interrupt on STM32.

5.4 Sequential Interfaces

5.4.1 UART (Universal Asynchronous Receiver/Transmitter)

UART is an asynchronous (no shared clock) serial protocol:

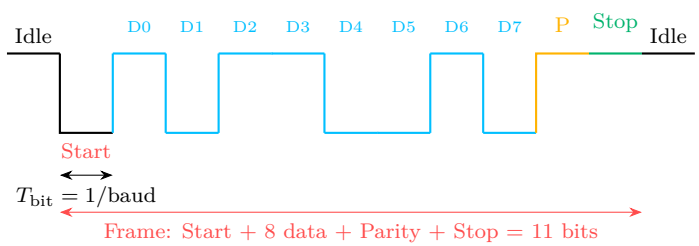

Figure 5.2: UART frame format: idle HIGH, start bit (LOW), 8 data bits (LSB first), optional parity, stop bit (HIGH).

Baud rate = bits per second. Common: 9600, 115200, 921600. RS-232 uses ±12 V signaling; TTL UART uses 0/3.3 V.

5.4.2 SPI (Serial Peripheral Interface)

SPI is a synchronous, full-duplex, 4-wire protocol:

- `MOSI`: Master Out, Slave In (data from master to slave).
- `MISO`: Master In, Slave Out (data from slave to master).
- `SCK`: Serial Clock (generated by master).
- `CS` (or `SS`): Chip Select (active low), selects the slave device.

Four SPI modes defined by clock polarity (CPOL) and clock phase (CPHA):

Table 5.2: SPI modes.

Mode	CPOL	CPHA	Description
0	0	0	Clock idle LOW, sample on rising edge
1	0	1	Clock idle LOW, sample on falling edge
2	1	0	Clock idle HIGH, sample on falling edge
3	1	1	Clock idle HIGH, sample on rising edge

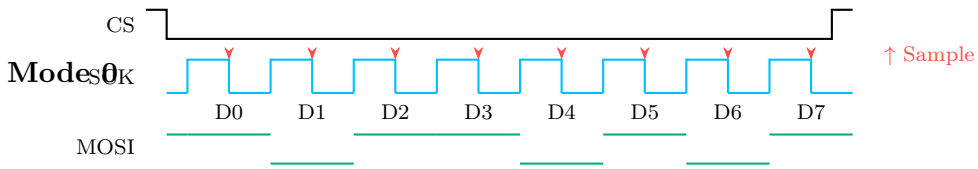

Figure 5.3: SPI Mode 0 timing: CPOL=0, CPHA=0. Data sampled on rising clock edge.

5.4.3 I²C (Inter-Integrated Circuit)

A 2-wire synchronous bus protocol:

- SCL: Serial Clock (master drives).
- SDA: Serial Data (bidirectional, open-drain with pull-up).
- 7-bit or 10-bit device addressing.
- Speeds: Standard mode (100 kbps), Fast mode (400 kbps), Fast mode+ (1 Mbps).

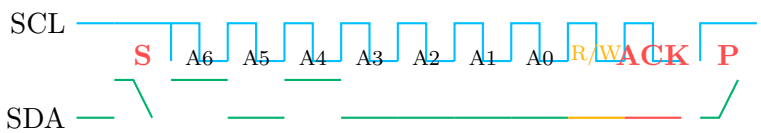

Figure 5.4: I²C transaction: START (S), 7-bit address, R/W bit, ACK from slave, STOP (P).

Multi-master arbitration: If two masters start transmitting simultaneously, arbitration is performed bit-by-bit on SDA. A master sending HIGH but reading LOW (another master is driving LOW) loses arbitration and backs off.

Clock stretching: A slave can hold SCL LOW to pause the master while it processes data.

5.4.4 Other Buses

- **1-Wire (Maxim/Dallas):** Single data wire + ground. Used for temperature sensors (DS18B20), iButtons. Parasitic power from the data line.
- **CAN (Controller Area Network):** Differential 2-wire bus for automotive and industrial. Multi-master with priority-based arbitration. Up to 1 Mbps (classic CAN) or 5 Mbps (CAN FD).

5.5 Code Examples

```c
// Transmit a byte over SPI1 (polling mode)
void spi_transmit_byte(uint8_t data) {
    // Wait until TX buffer is empty
    while (!(SPI1->SR & SPI_SR_TXE))
        ;
    // Write data to DR (starts transmission)
    *(volatile uint8_t *)&SPI1->DR = data;
    // Wait until transmission complete
    while (SPI1->SR & SPI_SR_BSY)
        ;
}

// Transmit/receive a byte (full duplex)
uint8_t spi_transfer(uint8_t tx_data) {
    while (!(SPI1->SR & SPI_SR_TXE))
        ;
    *(volatile uint8_t *)&SPI1->DR = tx_data;
    while (!(SPI1->SR & SPI_SR_RXNE))
        ;
    return *(volatile uint8_t *)&SPI1->DR;
}
```

Listing 5.3: STM32 HAL-style SPI transmit.

```c
#define SDA_HIGH() GPIOB->BSRR = (1U << 7)
#define SDA_LOW() GPIOB->BSRR = (1U << 23)
#define SCL_HIGH() GPIOB->BSRR = (1U << 6)
#define SCL_LOW() GPIOB->BSRR = (1U << 22)
#define SDA_READ() ((GPIOB->IDR >> 7) & 1U)

static void i2c_delay(void) {
    for (volatile int i = 0; i < 10; i++); // ~5 us at 72 MHz
}

void i2c_start(void) {
    SDA_HIGH(); SCL_HIGH(); i2c_delay();
    SDA_LOW(); i2c_delay(); // SDA falls while SCL is HIGH
    SCL_LOW(); i2c_delay();
}

void i2c_stop(void) {
    SDA_LOW(); SCL_HIGH(); i2c_delay();
    SDA_HIGH(); i2c_delay(); // SDA rises while SCL is HIGH
}

uint8_t i2c_write_byte(uint8_t data) {
```

```c
    for (int i = 7; i >= 0; i--) {
        if (data & (1 << i)) SDA_HIGH();
        else                 SDA_LOW();
        i2c_delay();
        SCL_HIGH(); i2c_delay();
        SCL_LOW(); i2c_delay();
    }
    // Read ACK (slave drives SDA LOW for ACK)
    SDA_HIGH(); // Release SDA for slave
    SCL_HIGH(); i2c_delay();
    uint8_t ack = !SDA_READ(); // ACK = SDA LOW
    SCL_LOW(); i2c_delay();
    return ack;
}
```

Listing 5.4: Bit-bang I^{2}C master in C (simplified).

5.6 Worked Examples

Example 5.1: UART Baud Rate Calculation

A UART module derives its baud rate from a 72 MHz peripheral clock using a 16× oversampling divider:

$$\text{USARTDIV} = \frac{f_{\text{CLK}}}{16 \times \text{Baud}}$$

Calculate the USARTDIV register value for 115200 baud and determine the actual baud rate and percentage error.

Solution

$$\text{USARTDIV} = \frac{72{,}000{,}000}{16 \times 115{,}200} = \frac{72{,}000{,}000}{1{,}843{,}200} = 39.0625$$

The register stores this as a fixed-point value: integer part = 39, fractional part = $0.0625 \times 16 = 1$.
BRR register value: $(39 \ll 4)|1 = 625 =$ `0x0271`.
Actual baud rate:

$$\text{Baud}_{\text{actual}} = \frac{72{,}000{,}000}{16 \times 39.0625} = \frac{72{,}000{,}000}{625} = 115{,}200$$

Error: exactly 0%. In this case, the clock divides perfectly.

Example 5.2: ADC Resolution and Accuracy

A 12-bit ADC with $V_{\text{ref}} = 3.3\,\text{V}$ measures a sensor output. Calculate: (a) the voltage resolution (LSB), (b) the digital output for $V_{\text{in}} = 1.65\,\text{V}$, (c) the quantisation error.

Solution

(a) Voltage resolution:

$$\text{LSB} = \frac{V_{\text{ref}}}{2^{12}} = \frac{3.3}{4096} = 0.806\,\text{mV}$$

(b) Digital output:

$$D = \left\lfloor \frac{1.65 \times 4096}{3.3} \right\rfloor = \lfloor 2048 \rfloor = 2048 = \texttt{0x800}$$

(c) Maximum quantisation error is $\pm 0.5\,\text{LSB} = \pm 0.403\,\text{mV}$.

Example 5.3: SPI Transfer Time

An SPI master communicates with a Flash memory at 18 MHz clock. To read 4096 bytes, the master sends a 1-byte command + 3-byte address, then reads 4096 bytes. Calculate total transfer time.

Solution

Total bytes transferred: $1 + 3 + 4096 = 4100$ bytes $= 32{,}800$ bits.
Time per bit: $1/18\,\text{MHz} = 55.6\,\text{ns}$.
Total transfer time: $32{,}800 \times 55.6\,\text{ns} = 1.823\,\text{ms}$.
Effective throughput: $4096/1.823\,\text{ms} = 2.25\,\text{MB/s}$. (Overhead from command and address is small.)

Example 5.4: I²C Transaction Length

Calculate the time to write 2 data bytes to an I²C slave at 400 kbps (Fast mode). Include START, address, ACK bits, and STOP.

Solution

Bits on the bus:

- START condition: 1 bit period
- Address (7 bits) + R/W (1 bit) = 8 bits
- ACK from slave: 1 bit
- Data byte 1: 8 bits + ACK: 1 bit = 9 bits
- Data byte 2: 8 bits + ACK: 1 bit = 9 bits
- STOP condition: 1 bit period

Total: $1 + 8 + 1 + 9 + 9 + 1 = 29$ bit periods.
At 400 kbps: $T = 29/400{,}000 = 72.5\,\mu\text{s}$.

5.7 Practice Problems

Exercise 5.1: GPIO Configuration

On an STM32F4, configure PA0 as analog input for the ADC (no pull-up/pull-down) and PA5 as alternate function push-pull for SPI1_SCK using direct register writes.

Exercise 5.2: ADC Sampling Design

A vibration sensor outputs frequencies up to 5 kHz. Design the ADC system: sampling rate, anti-aliasing filter cutoff frequency, and ADC resolution needed to achieve 60 dB SNR.

Exercise 5.3: Protocol Selection

For each scenario, choose the most appropriate serial protocol (UART, SPI, I^2C, or CAN) and justify:

(a) Reading a temperature sensor that updates once per second.

(b) Streaming audio samples at 48 kHz, 16-bit stereo.

(c) Networking 20 sensor nodes in a factory over 100 m total cable length.

(d) High-speed communication with an external SPI Flash memory.

Exercise 5.4: SPI Timing Analysis

Draw the complete SPI Mode 2 (CPOL=1, CPHA=0) timing diagram for transmitting the byte `0xA5` (MSB first): show SCK, MOSI, MISO (assume slave returns `0x3C`), and CS waveforms.

Exercise 5.5: DMA vs. Interrupt Comparison

An application receives UART data at 921600 baud (8N1). Calculate the interrupt rate and CPU overhead with interrupt-driven reception. Then propose a DMA-based alternative and estimate the reduction in CPU load.

Exercise 5.6: PWM Motor Control

A DC motor requires PWM at 20 kHz with 0.1% duty cycle resolution. The timer clock is 168 MHz. Calculate the required timer prescaler and auto-reload register values.

Chapter 6

Multitasking

Module: 6 **Lecture Hours:** 2 **ILOs Addressed:** ILO-4, ILO-5

Learning Outcomes: By the end of this module, students will be able to:

1. Explain the limitations of super-loop architectures and motivate multitasking.
2. Describe processes, threads, and task state transitions.
3. Explain context switching mechanics on ARM Cortex-M processors.
4. Implement inter-task communication using mutexes, semaphores, and queues.
5. Write FreeRTOS task code with synchronisation primitives.

6.1 Imperative Programs — Single Task Baseline

6.1.1 Super-Loop Architecture

The simplest embedded software architecture is the "super-loop" or "bare-metal while(1)":

```c
int main(void) {
    system_init();
    while (1) {
        read_sensors();
        process_data();
        update_display();
        handle_communication();
    }
}
```

Listing 6.1: Typical super-loop architecture.

Advantages: Simple, no OS overhead, deterministic if well-designed, minimal RAM usage.

Disadvantages:

- **Latency:** Response time to an event depends on the execution time of all other tasks in the loop. If `process_data()` takes 50 ms, sensor reading is delayed by up to 50 ms.

- **Priority:** All "tasks" have equal priority; no way to preempt a low-priority task for a high-priority event.
- **Scalability:** Adding new functionality increases loop period and worsens latency.

6.1.2 State Machine Approach

A structured alternative partitions each task into states, executing one state per loop iteration:

```c
typedef enum { IDLE, SAMPLING, PROCESSING, DONE } sensor_state_t;
sensor_state_t sensor_state = IDLE;

void sensor_tick(void) {
    switch (sensor_state) {
        case IDLE:
            if (timer_expired()) sensor_state = SAMPLING;
            break;
        case SAMPLING:
            start_adc();
            sensor_state = PROCESSING;
            break;
        case PROCESSING:
            if (adc_done()) {
                result = get_adc_value();
                sensor_state = DONE;
            }
            break;
        case DONE:
            send_result(result);
            sensor_state = IDLE;
            break;
    }
}
```

Listing 6.2: State machine approach in super-loop.

This improves responsiveness (each state executes quickly) but makes code complex and fragile as the number of tasks grows.

6.2 Processes and Threads

Definition 6.1: Process

A process is an instance of a running program with its own **private address space**. Processes are isolated from each other; inter-process communication (IPC) requires explicit mechanisms (pipes, shared memory, message queues). Context switching between processes is expensive because it involves switching page tables (on systems with an MMU).

Definition 6.2: Thread

A thread is a lightweight unit of execution within a process. Threads within the same process **share the same address space** (code, global data, heap) but have **individual stacks and register contexts**. Context switching between threads is faster (no page table switch).

In embedded RTOS terminology, a "task" is synonymous with a thread. Most embedded RTOSs (FreeRTOS, Zephyr, RTX) use a single address space with multiple tasks/threads.

△ Key Point: Race Conditions

Because threads share memory, concurrent access to shared variables without synchronisation leads to **race conditions**—bugs where the program output depends on the unpredictable timing of thread execution. This is a primary source of embedded software defects.

6.2.1 Thread States

A thread (task) transitions through the following states:

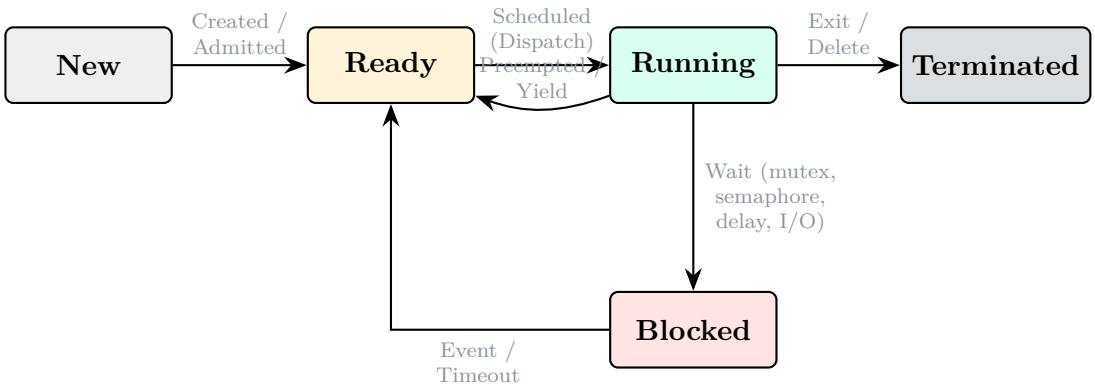

Figure 6.1: Thread (task) state transition diagram: New → Ready → Running ↔ Blocked → Terminated.

- **New:** Task created but not yet added to the scheduler.
- **Ready:** Task is eligible to run; waiting for the CPU.

- **Running:** Task is currently executing on the CPU (only one task at a time on single-core).
- **Blocked:** Task is waiting for an event (mutex, semaphore, queue message, or delay). Cannot be scheduled.
- **Terminated:** Task has completed or been deleted; resources freed.

6.3 Context Switching Mechanics

Definition 6.3: Context Switch

A context switch saves the complete execution state (PC, SP, registers, status flags) of the currently running task and restores the saved state of the next task to be run. On ARM Cortex-M, this is typically performed by the **PendSV** exception handler.

6.3.1 What Is Saved

On ARM Cortex-M, the context consists of:

- **Hardware-saved (automatically on exception entry):** R0–R3, R12, LR, PC, xPSR (8 registers, 32 bytes).
- **Software-saved (by PendSV handler):** R4–R11 (8 registers, 32 bytes). If FPU is used: S16–S31 + FPSCR (additional 68 bytes).

6.3.2 ARM Cortex-M: PSP vs. MSP

ARM Cortex-M has two stack pointers:

- **MSP (Main Stack Pointer):** Used by exception handlers and privileged code (the OS kernel).
- **PSP (Process Stack Pointer):** Used by tasks in thread mode. Each task has its own PSP pointing to its private stack.

6.3.3 PendSV Context Switch

```
    .global PendSV_Handler
    .type PendSV_Handler, %function
PendSV_Handler:
    ; --- Save current task context ---
    MRS     R0, PSP             ; Get current task's stack pointer
    STMDB   R0!, {R4-R11}       ; Push R4-R11 onto task stack
    ; (Hardware already pushed R0-R3, R12, LR, PC, xPSR)

    ; Save updated PSP to current TCB
    LDR     R1, =pxCurrentTCB
```

```
    LDR     R2, [R1]            ; R2 = address of current TCB
    STR     R0, [R2]            ; TCB->pxTopOfStack = R0

    ; --- Select next task ---
    PUSH    {R3, LR}
    BL      vTaskSwitchContext ; Call scheduler to pick next task
    POP     {R3, LR}

    ; --- Restore next task context ---
    LDR     R1, =pxCurrentTCB
    LDR     R2, [R1]            ; R2 = address of next TCB
    LDR     R0, [R2]            ; R0 = next task's stack pointer

    LDMIA   R0!, {R4-R11}      ; Pop R4-R11 from next task's stack
    MSR     PSP, R0             ; Set PSP to next task's stack

    BX      LR                  ; Return (hardware restores R0-R3 etc.)
```

Listing 6.3: Simplified PendSV handler for FreeRTOS-style context switch on Cortex-M4.

6.3.4 Context Switch Overhead

On Cortex-M4 at 168 MHz:

- Hardware register stacking: 12 cycles.
- Software save (R4–R11): 8 cycles (STMDB).
- Scheduler call: 20–50 cycles (depends on ready queue implementation).
- Software restore: 8 cycles (LDMIA).
- Hardware unstacking: 12 cycles.
- **Total:** $\approx$ 60–90 cycles = 0.36–0.54 μs at 168 MHz.

▶ Practical Note

FreeRTOS typically measures 80–100 cycles per context switch on Cortex-M4. With FPU context (lazy stacking disabled), this increases to ~180 cycles. Using FPU lazy stacking (default in FreeRTOS), the FPU context is only saved when a second task uses the FPU.

6.4 Inter-Task Communication

6.4.1 Shared Memory with Mutexes

Definition 6.4: Mutex

A mutex (mutual exclusion) is a synchronisation primitive that ensures only one task can access a shared resource at a time. A task must *acquire* the mutex before accessing the resource and *release* it afterward.

Deadlock: Occurs when two or more tasks are each waiting for a mutex held by another, forming a circular dependency. Example: Task A holds Mutex 1 and waits for Mutex 2; Task B holds Mutex 2 and waits for Mutex 1.

Priority inversion: A high-priority task is blocked waiting for a mutex held by a low-priority task, while a medium-priority task preempts the low-priority task, effectively inverting priorities.

Priority inheritance: Solution where the mutex-holding low-priority task temporarily inherits the priority of the highest-priority waiting task, preventing the medium-priority task from preempting it.

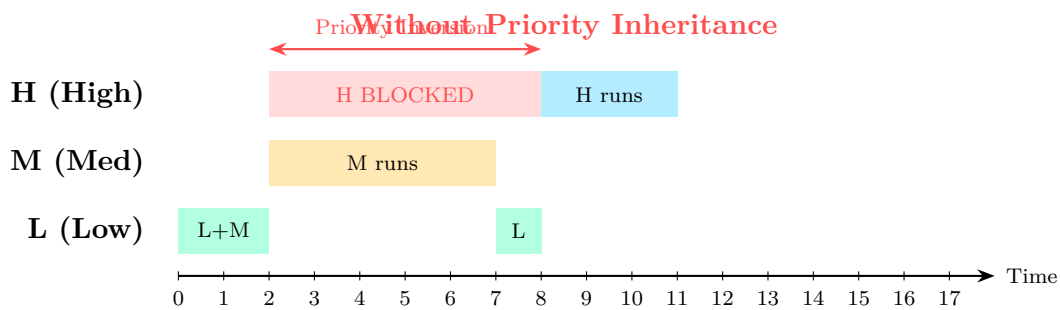

Figure 6.2: Priority inversion: high-priority task H is blocked while medium-priority task M runs.

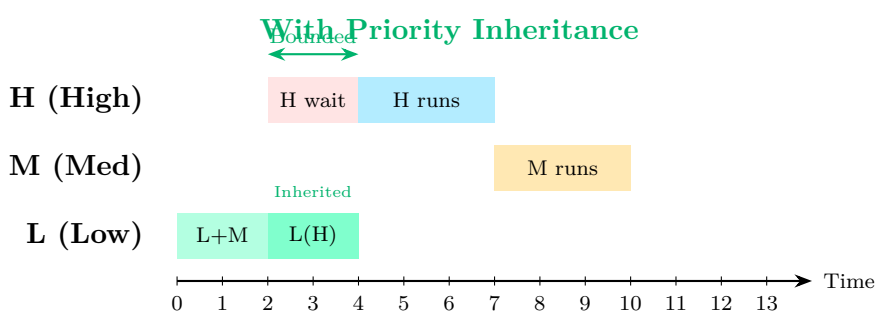

Figure 6.3: Priority inheritance: L inherits H's priority, preventing M from causing unbounded inversion.

6.4.2 Message Passing: Queues and Mailboxes

Queues provide safe, buffered communication between tasks without shared memory:

- **FIFO queue:** First-In-First-Out. Producer enqueues, consumer dequeues.
- **Priority queue:** Messages dequeued by priority rather than arrival order.
- **Mailbox:** Single-item queue (the latest message overwrites the previous one).

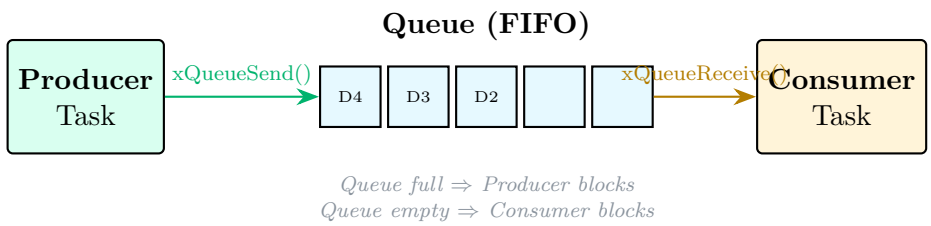

Figure 6.4: Producer-consumer pattern using a FIFO queue for inter-task communication.

6.4.3 Semaphores

- **Binary semaphore:** Value is 0 or 1. Used for signalling between tasks (e.g., ISR signals task that data is ready). Similar to mutex but does not support priority inheritance.
- **Counting semaphore:** Value is 0 to N. Used to control access to a pool of N identical resources or to count events.

6.4.4 Event Flags / Event Groups

Event groups allow a task to wait for a combination of events (logical AND or OR of multiple flags). FreeRTOS: `xEventGroupWaitBits()` blocks until the specified combination of bits is set.

6.5 FreeRTOS as a Concrete Example

```c
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

SemaphoreHandle_t xMutex;
volatile uint32_t shared_counter = 0;

void vTaskIncrement(void *pvParameters) {
    (void)pvParameters;
    for (;;) {
        if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE) {
            // Critical section: safely modify shared resource
            shared_counter++;
            xSemaphoreGive(xMutex);
        }
        vTaskDelay(pdMS_TO_TICKS(100)); // Delay 100 ms
    }
}

void vTaskDisplay(void *pvParameters) {
```

```c
    (void)pvParameters;
    for (;;) {
        if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE) {
            uint32_t val = shared_counter;
            xSemaphoreGive(xMutex);
            // Use val safely outside critical section
            printf("Counter: %lu\n", val);
        }
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

int main(void) {
    xMutex = xSemaphoreCreateMutex();

    xTaskCreate(vTaskIncrement, "Inc", 256, NULL, 2, NULL);
    xTaskCreate(vTaskDisplay, "Disp", 256, NULL, 1, NULL);

    vTaskStartScheduler(); // Never returns
    for (;;);
}
```

Listing 6.4: FreeRTOS: two tasks sharing a resource with a mutex.

Key FreeRTOS APIs:

- `xTaskCreate(function, name, stackSize, params, priority, handle)`: Creates a new task.
- `xSemaphoreCreateMutex()`: Creates a mutex with priority inheritance.
- `xQueueCreate(length, itemSize)`: Creates a queue.
- `xEventGroupCreate()`: Creates an event group.
- `vTaskDelay(ticks)`: Blocks the task for the specified number of tick periods.
- `vTaskDelayUntil(&xLastWakeTime, period)`: Precise periodic execution.

6.6 Worked Examples

Example 6.1: Context Switch Time Budget

An RTOS runs on Cortex-M4 at 168 MHz with a 1 ms tick period. If context switching takes 100 cycles, what percentage of CPU time is consumed by context switches if 10 switches occur per tick period?

Solution

Cycles per tick: $168 \times 10^6 \times 10^{-3} = 168{,}000$ cycles.
Cycles for switching: $10 \times 100 = 1{,}000$ cycles.
Overhead: $\frac{1{,}000}{168{,}000} = 0.595\%$.
This is negligible—less than 1% overhead, which is why RTOS-based multitasking is practical.

Example 6.2: Deadlock Identification

Two tasks access two shared resources (UART and SPI) protected by mutexes A and B:

```
// Task 1:          // Task 2:
take(mutexA);       take(mutexB);
take(mutexB);       take(mutexA);
// use UART+SPI     // use SPI+UART
give(mutexB);       give(mutexA);
give(mutexA);       give(mutexB);
```

Identify the deadlock potential and propose a fix.

Solution

Deadlock scenario: Task 1 acquires mutexA, then is preempted. Task 2 acquires mutexB. Now Task 1 waits for mutexB (held by Task 2) and Task 2 waits for mutexA (held by Task 1). Circular wait = deadlock.
Fix: Impose a global lock ordering: always acquire mutexA before mutexB. Both tasks use the same order:

```
// Both tasks:
take(mutexA);
take(mutexB);
// use resources
give(mutexB);
give(mutexA);
```

This eliminates the circular dependency.

Example 6.3: Stack Size Estimation

A FreeRTOS task on Cortex-M4 calls functions with a maximum call depth of 6 levels, each using 32 bytes of local variables. The context save frame is 64 bytes (R0–R11, LR, PC, xPSR, padding). Estimate the minimum stack size.

Solution

- Context frame: 64 bytes
- Function call overhead (return address per level): $6 \times 4 = 24$ bytes
- Local variables: $6 \times 32 = 192$ bytes
- Safety margin (recommended 25%): $0.25 \times 280 = 70$ bytes

Total: $64 + 24 + 192 + 70 = 350$ bytes. Round up to **384 bytes** (96 words).
In FreeRTOS, stack size is specified in words: `xTaskCreate(..., 96, ...)`.

6.7 Practice Problems

Exercise 6.1: Super-Loop Timing Analysis

A super-loop executes three tasks: Task A (2 ms), Task B (5 ms), Task C (3 ms). Calculate the worst-case response time for each task. How would adding a 50 ms Task D affect Task A's response time?

Exercise 6.2: Process vs. Thread

Explain why embedded RTOSs for Cortex-M (which lack an MMU) use threads rather than processes. What security/safety risk does a single address space introduce?

Exercise 6.3: Priority Inversion Scenario

Three tasks: H (priority 3), M (priority 2), L (priority 1). L holds a mutex needed by H. Draw Gantt charts showing: (a) execution without priority inheritance, (b) execution with priority inheritance. Assume L needs 4 time units in the critical section and M arrives at time 2.

Exercise 6.4: FreeRTOS Queue Usage

Write FreeRTOS code for a producer task that reads ADC values every 10 ms and sends them to a consumer task via a 32-element queue. The consumer calculates a running average and prints it every 500 ms.

Exercise 6.5: Stack Overflow Detection

Describe two methods FreeRTOS provides for detecting stack overflows at runtime. What are the limitations of each method? How would you determine the optimal stack size for a task during development?

Chapter 7

Task Scheduling

Module: 7 **Lecture Hours:** 2 **ILOs Addressed:** ILO-4, ILO-5

Learning Outcomes: By the end of this module, students will be able to:

1. Classify scheduling algorithms as static vs. dynamic, preemptive vs. cooperative.
2. Apply Rate Monotonic Scheduling (RMS) and Earliest Deadline First (EDF) to a task set.
3. Perform schedulability analysis using the utilisation bound test.
4. Construct Gantt charts for given task sets.
5. Discuss multiprocessor scheduling challenges and WCET estimation.

7.1 Scheduling Fundamentals

Definition 7.1: Scheduling Algorithm

A scheduling algorithm determines, at each scheduling decision point, which ready task to execute next. The scheduler's goal is to ensure **all tasks meet their deadlines** while maximising CPU utilisation.

7.1.1 Task Model

A periodic real-time task τ_i is characterised by:

- C_i: Worst-case execution time (WCET)
- T_i: Period
- D_i: Relative deadline (typically $D_i = T_i$ for implicit-deadline tasks)

The **CPU utilisation** due to task τ_i is:

$$U_i = \frac{C_i}{T_i}$$

Total utilisation for n tasks:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i}$$

△ Key Point: Necessary Condition for Schedulability

For any scheduling algorithm on a single processor: $U \leq 1$. If $U > 1$, no algorithm can schedule the task set without missing deadlines.

7.1.2 Classification of Schedulers

Table 7.1: Classification of scheduling algorithms.

Criterion	Options
Priority assignment	*Static* (fixed at design time) vs. *Dynamic* (changes at runtime)
Preemption	*Preemptive* (running task can be interrupted) vs. *Non-preemptive* (runs to completion or yield)
Trigger	*Clock-driven* (decisions at tick) vs. *Event-driven* (decisions on task arrival/completion)

7.2 Rate Monotonic Scheduling (RMS)

Definition 7.2: Rate Monotonic Scheduling

RMS is a **static-priority, preemptive** scheduling algorithm where priorities are assigned inversely proportional to the period: the shorter the period, the higher the priority. RMS is **optimal** among fixed-priority algorithms for implicit-deadline periodic tasks (Liu & Layland, 1973 [1]).

7.2.1 RMS Utilisation Bound

For n implicit-deadline tasks, RMS guarantees schedulability if:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq n\left(2^{1/n} - 1\right)$$

Table 7.2: Liu–Layland utilisation bound $U_{\text{lub}}(n) = n(2^{1/n} - 1)$.

n	1	2	3	4	5	10	∞
U_{lub}	1.000	0.828	0.780	0.757	0.743	0.718	$\ln 2 = 0.693$

◇ ***Insight***

The utilisation bound converges to $\ln 2 \approx 0.693$ *as* $n \to \infty$. *This is a* sufficient *condition: if* $U \leq U_{\text{lub}}$, *the task set is schedulable under RMS. If* $U_{\text{lub}} < U \leq 1$, *the task set* may or may not *be schedulable — further analysis (e.g., response time*

analysis) is needed.

7.2.2 Response Time Analysis (RTA)

When the utilisation bound test is inconclusive, exact schedulability can be determined by computing the worst-case response time R_i for each task:

$$R_i = C_i + \sum_{j \in \mathrm{hp}(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j$$

where $\mathrm{hp}(i)$ is the set of tasks with higher priority than τ_i. This is solved iteratively:

$$R_i^{(k+1)} = C_i + \sum_{j \in \mathrm{hp}(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j$$

starting from $R_i^{(0)} = C_i$, until convergence or $R_i > D_i$.

7.3 Earliest Deadline First (EDF)

Definition 7.3: Earliest Deadline First

EDF is a **dynamic-priority, preemptive** scheduling algorithm that always schedules the ready task with the nearest absolute deadline. EDF is **optimal** among all uniprocessor scheduling algorithms for preemptive, independent tasks.

Schedulability test: For implicit-deadline tasks under EDF:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq 1$$

This is both necessary and sufficient—EDF can fully utilise the processor.

7.3.1 Comparison: RMS vs. EDF

Table 7.3: Comparison of RMS and EDF scheduling algorithms.

Property	RMS	EDF
Priority type	Static (fixed at design)	Dynamic (changes at runtime)
Optimality	Optimal among fixed-priority	Optimal among all uniprocessor
Util. bound	$n(2^{1/n} - 1) \to \ln 2$	1.0 (full utilisation)
Overload behaviour	Predictable (lowest-priority task misses first)	Unpredictable (domino effect)
Implementation	Simple (priority comparison)	Moderate (deadline tracking)
Jitter	Higher for low-priority tasks	Lower and more uniform
Industry adoption	Widely used (VxWorks, FreeRTOS)	Less common in safety-critical

7.4 Deadline Monotonic Scheduling (DMS)

Definition 7.4: Deadline Monotonic Scheduling

DMS is a **static-priority** algorithm where priorities are assigned inversely proportional to *relative deadlines*: the shorter the deadline, the higher the priority. DMS is optimal among fixed-priority algorithms for *constrained-deadline* tasks ($D_i \leq T_i$). For implicit-deadline tasks ($D_i = T_i$), DMS reduces to RMS.

7.5 Gantt Charts

Gantt charts visualise the schedule by showing which task occupies the CPU at each time instant.

Example 7.1: RMS Gantt Chart

Given tasks: $\tau_1 = (C_1 = 1, T_1 = 4)$, $\tau_2 = (C_2 = 2, T_2 = 6)$, $\tau_3 = (C_3 = 3, T_3 = 12)$. Check schedulability and draw the RMS Gantt chart for the hyperperiod.

Solution

Step 1: Assign priorities. RMS: shorter period = higher priority.

$$\text{Priority: } \tau_1 > \tau_2 > \tau_3$$

Step 2: Calculate utilisation.

$$U = \frac{1}{4} + \frac{2}{6} + \frac{3}{12} = 0.250 + 0.333 + 0.250 = 0.833$$

Utilisation bound for $n = 3$: $U_{\text{lub}} = 3(2^{1/3} - 1) = 0.780$.
Since $U = 0.833 > 0.780$, the utilisation bound test is *inconclusive*. We must perform detailed analysis.

Step 3: Response Time Analysis.

- $R_1 = C_1 = 1 \leq D_1 = 4$ ✓
- R_2: $R_2^{(0)} = 2$; $R_2^{(1)} = 2 + \lceil 2/4 \rceil \times 1 = 2 + 1 = 3$; $R_2^{(2)} = 2 + \lceil 3/4 \rceil \times 1 = 2 + 1 = 3$. Converged: $R_2 = 3 \leq 6$ ✓
- R_3: $R_3^{(0)} = 3$; $R_3^{(1)} = 3 + \lceil 3/4 \rceil \times 1 + \lceil 3/6 \rceil \times 2 = 3 + 1 + 2 = 6$; $R_3^{(2)} = 3 + \lceil 6/4 \rceil \times 1 + \lceil 6/6 \rceil \times 2 = 3 + 2 + 2 = 7$; $R_3^{(3)} = 3 + \lceil 7/4 \rceil \times 1 + \lceil 7/6 \rceil \times 2 = 3 + 2 + 4 = 9$; $R_3^{(4)} = 3 + \lceil 9/4 \rceil \times 1 + \lceil 9/6 \rceil \times 2 = 3 + 3 + 4 = 10$; $R_3^{(5)} = 3 + \lceil 10/4 \rceil \times 1 + \lceil 10/6 \rceil \times 2 = 3 + 3 + 4 = 10$. Converged: $R_3 = 10 \leq 12$ ✓

All tasks schedulable!

Step 4: Gantt chart. Hyperperiod: $\text{lcm}(4, 6, 12) = 12$.

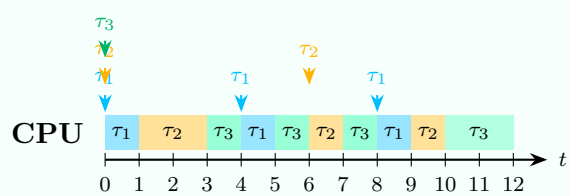

Figure 7.1: RMS Gantt chart for the task set over the hyperperiod $[0, 12)$.

Example 7.2: EDF Gantt Chart

For the same task set $\tau_1 = (1, 4)$, $\tau_2 = (2, 6)$, $\tau_3 = (3, 12)$, draw the EDF schedule.

Solution

Under EDF, absolute deadlines determine priority at each instant.

- $t = 0$: Deadlines: $\tau_1 : 4$, $\tau_2 : 6$, $\tau_3 : 12$. Run τ_1 (deadline 4).
- $t = 1$: τ_1 done. Deadlines: $\tau_2 : 6$, $\tau_3 : 12$. Run τ_2.
- $t = 3$: τ_2 done. Run τ_3.
- $t = 4$: τ_1 arrives (deadline 8). τ_3 has deadline 12. Preempt: run τ_1.
- $t = 5$: τ_1 done. Resume τ_3.
- $t = 6$: τ_2 arrives (deadline 12). τ_3 has remaining 1 unit, deadline 12; τ_2 deadline 12 (tie). Continue τ_3.
- $t = 7$: τ_3 done. Run τ_2.
- $t = 8$: τ_1 arrives (deadline 12). τ_2 has 1 unit remaining, deadline 12. Continue τ_2.
- $t = 9$: τ_2 done. Run τ_1.
- $t = 10$: τ_1 done. Idle or τ_3 second instance.

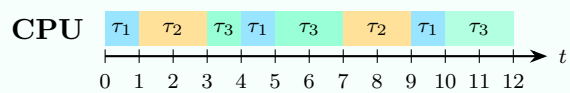

Figure 7.2: EDF schedule for the same task set. Note the different execution order due to dynamic priorities.

Both RMS and EDF succeed for this task set. EDF distributes slack more evenly.

Example 7.3: Schedulability Failure under RMS

Tasks: $\tau_1 = (C_1 = 2, T_1 = 5)$, $\tau_2 = (C_2 = 4, T_2 = 7)$. Check RMS schedulability.

Solution

$U = 2/5 + 4/7 = 0.400 + 0.571 = 0.971$.
$U_{\text{lub}}(2) = 2(\sqrt{2} - 1) = 0.828$.
Since $U = 0.971 > 0.828$, the bound test fails. Perform RTA:
τ_1**:** $R_1 = C_1 = 2 \leq 5$ ✓
τ_2**:** $R_2^{(0)} = 4$; $R_2^{(1)} = 4 + \lceil 4/5 \rceil \times 2 = 4 + 2 = 6$; $R_2^{(2)} = 4 + \lceil 6/5 \rceil \times 2 = 4 + 4 = 8$; $R_2^{(3)} = 4 + \lceil 8/5 \rceil \times 2 = 4 + 4 = 8$. Converged: $R_2 = 8 > D_2 = 7$ ✗
τ_2 misses its deadline under RMS!
Under EDF: $U = 0.971 \leq 1.0$, so EDF can schedule this task set. This demonstrates EDF's superior utilisation capability.

Example 7.4: RMS with Blocking

A system has three tasks. Task τ_3 (lowest priority) holds a mutex for 1 time unit that τ_1 (highest priority) needs. With priority inheritance: calculate R_1 accounting for

blocking.
Task set: $\tau_1 = (C_1 = 1, T_1 = 5)$, $\tau_2 = (C_2 = 2, T_2 = 8)$, $\tau_3 = (C_3 = 3, T_3 = 15)$, blocking time $B_1 = 1$ (critical section of τ_3).

Solution

The response time equation with blocking becomes:

$$R_i = C_i + B_i + \sum_{j \in \mathrm{hp}(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j$$

For τ_1: $R_1^{(0)} = C_1 + B_1 = 1 + 1 = 2$. No higher-priority tasks, so $R_1 = 2 \leq 5$ ✓.
For τ_2 (no blocking from τ_3's mutex, as τ_2 doesn't use it): $R_2^{(0)} = 2$; $R_2^{(1)} = 2 + \lceil 2/5 \rceil \times 1 = 2 + 1 = 3$; $R_2^{(2)} = 2 + \lceil 3/5 \rceil = 3$. Converged: $R_2 = 3 \leq 8$ ✓.
For τ_3: $R_3^{(0)} = 3$; $R_3^{(1)} = 3 + \lceil 3/5 \rceil (1) + \lceil 3/8 \rceil (2) = 3 + 1 + 2 = 6$; $R_3^{(2)} = 3 + \lceil 6/5 \rceil (1) + \lceil 6/8 \rceil (2) = 3+2+2 = 7$; $R_3^{(3)} = 3 + \lceil 7/5 \rceil (1) + \lceil 7/8 \rceil (2) = 3+2+2 = 7$.
Converged: $R_3 = 7 \leq 15$ ✓.
All tasks meet deadlines even with blocking.

7.6 Multiprocessor Scheduling

For systems with m processors, two approaches exist:

- **Partitioned scheduling:** Each task is statically assigned to one processor. Each processor uses a uniprocessor scheduler (RMS or EDF). No migration. Simple to implement.
- **Global scheduling:** All tasks share a single ready queue. The m highest-priority ready tasks run on the m processors. Tasks may migrate between processors. Better utilisation but higher overhead and harder analysis.

△ Key Point: Dhall's Effect

Under global EDF on m processors, utilisation can be as low as $\frac{m}{2m-1}$ (about 50% for large m) while still missing deadlines. This counter-intuitive result shows that uniprocessor optimality does not extend to multiprocessors.

7.7 Worst-Case Execution Time (WCET)

Accurate WCET estimation is critical for schedulability analysis.

Methods:

- **Measurement-based:** Run the task many times, record the maximum. Risk: may miss the true worst case.

- **Static analysis:** Analyse the binary's control flow and processor timing model (IPET — Implicit Path Enumeration Technique). Conservative but safe. Tools: aiT, Bound-T.
- **Hybrid:** Combine measurement of basic blocks with static analysis of paths.

WCET challenges on modern processors:

- Branch prediction (misprediction penalties)
- Cache effects (cache misses add tens of cycles)
- Pipeline stalls (data hazards, structural hazards)
- On multi-core: shared bus/cache contention

7.8 Practice Problems

Exercise 7.1: Basic Schedulability

Tasks: $\tau_1 = (C_1 = 1, T_1 = 3)$, $\tau_2 = (C_2 = 1, T_2 = 5)$, $\tau_3 = (C_3 = 2, T_3 = 15)$. (a) Compute the total utilisation. (b) Check schedulability under RMS using the utilisation bound. (c) Verify using RTA if the bound test fails. (d) Draw the Gantt chart for one hyperperiod under RMS.

Exercise 7.2: EDF vs. RMS

Tasks: $\tau_1 = (C_1 = 3, T_1 = 6)$, $\tau_2 = (C_2 = 3, T_2 = 8)$. (a) Show that this task set fails the RMS utilisation bound. (b) Determine using RTA whether it is actually schedulable under RMS. (c) Verify that it is schedulable under EDF. (d) Draw both Gantt charts.

Exercise 7.3: Response Time Analysis

Task set: $\tau_1 = (1, 4)$, $\tau_2 = (2, 5)$, $\tau_3 = (3, 20)$. An ISR adds blocking time $B_3 = 1$ to τ_3. Compute R_1, R_2, and R_3 using RTA with blocking. Is the system schedulable?

Exercise 7.4: Multiprocessor Partitioning

Given 5 tasks with utilisations $\{0.3, 0.4, 0.5, 0.3, 0.4\}$ and 2 processors, partition the tasks using First-Fit Decreasing. What is the utilisation of each processor? Would global EDF be able to schedule this set?

Exercise 7.5: WCET Impact

A task has measured execution times of $\{120, 135, 128, 142, 131\}\,\mu$s over 1000 runs. (a) What WCET would you use for static analysis? (b) If the static analysis tool reports $180\,\mu$s, discuss the implications of using $142\,\mu$s vs. $180\,\mu$s for schedulability analysis.

Exercise 7.6: Scheduler Design Trade-Off

A safety-critical automotive system controls ABS braking (deadline 2 ms) and infotainment display updates (deadline 100 ms). Argue whether RMS or EDF is more appropriate for this system, considering overload behaviour, certification requirements, and implementation complexity.

Chapter 8

Analysis and Verification of Embedded Systems

Module: 8 **Lecture Hours:** 4 **ILOs Addressed:** ILO-6, ILO-7

Learning Outcomes: By the end of this module, students will be able to:

1. Profile embedded software for execution time, memory usage, and power consumption.
2. Perform Failure Mode and Effects Analysis (FMEA) on embedded systems.
3. Calculate reliability metrics (MTTF, MTBF, availability, FIT rate).
4. Apply formal verification concepts (model checking with SPIN, temporal logic).
5. Explain MISRA-C guidelines and their role in safety-critical software.

8.1 Performance Analysis and Optimisation

8.1.1 Profiling Metrics

Key performance indicators for embedded systems:

- **Execution time:** CPU cycles or wall-clock time consumed by a function or task.
- **Memory footprint:** Flash (code + constants) and RAM (stack + heap + global data).
- **Power consumption:** Average and peak current draw; energy per operation.
- **Throughput:** Data processed per unit time (bytes/s, samples/s).
- **Latency:** Time from stimulus to response (interrupt latency, task response time).

8.1.2 ARM Cortex-M DWT Cycle Counter

The Data Watchpoint and Trace (DWT) unit on Cortex-M3/M4/M7 provides a free-running cycle counter.

```c
#include <stdint.h>

// DWT register addresses
#define DWT_CYCCNT (*(volatile uint32_t *)0xE0001004)
#define DWT_CONTROL (*(volatile uint32_t *)0xE0001000)
```

```c
#define SCB_DEMCR  (*(volatile uint32_t *)0xE000EDFC)

void dwt_init(void) {
    SCB_DEMCR |= (1 << 24);  // Enable DWT
    DWT_CYCCNT = 0;           // Reset counter
    DWT_CONTROL |= 1;         // Enable CYCCNT
}

static inline uint32_t dwt_get_cycles(void) {
    return DWT_CYCCNT;
}

// Usage example
void measure_function(void) {
    uint32_t start = dwt_get_cycles();

    // --- Code under measurement ---
    my_dsp_function();
    // ---

    uint32_t end = dwt_get_cycles();
    uint32_t elapsed = end - start; // handles wrap-around

    // At 168 MHz: time_us = elapsed / 168
    float time_us = (float)elapsed / 168.0f;
}
```

Listing 8.1: Using the DWT cycle counter for execution time measurement.

8.1.3 Memory Analysis

```
/* After building, use arm-none-eabi-size: */
//    text    data     bss     dec     hex filename
//   24576    1024    8192   33792    8400 firmware.elf
//
// Flash usage: text + data = 24576 + 1024 = 25600 bytes
// RAM usage:  data + bss = 1024 + 8192 = 9216 bytes
```

Listing 8.2: Checking flash and RAM usage from the linker map file.

8.1.4 Optimisation Techniques

Table 8.1: Common embedded optimisation strategies.

Technique	Description
Compiler optimisation flags	`-O2` (speed), `-Os` (size), `-Og` (debug-friendly)
Loop unrolling	Reduce loop overhead at the cost of code size
Lookup tables	Replace computation with pre-computed table (e.g., sine LUT)
Fixed-point arithmetic	Avoid floating-point on cores without FPU
DMA transfers	Offload data movement from CPU
Sleep modes	Reduce power by entering low-power states when idle
Dead code elimination	Remove unused functions (`-ffunction-sections -Wl,-gc-sections`)

▶ Practical Note

Always profile before optimising. Premature optimisation is the root of all evil (Knuth). Use the DWT counter or an oscilloscope to identify hotspots.

8.2 Failure Mode and Effects Analysis (FMEA)

Definition 8.1: FMEA

Failure Mode and Effects Analysis is a systematic, proactive method for analysing potential failure modes of a system, their causes, effects, and criticality. The output is a prioritised list of risks, each assigned a Risk Priority Number (RPN).

$$\text{RPN} = \text{Severity} \times \text{Occurrence} \times \text{Detection}$$

where each factor is rated 1–10 (1 = best, 10 = worst).

Table 8.2: Example FMEA table for an embedded temperature controller.

Component	Failure Mode	Effect	S	O	D	RPN	Mitigation
ADC	Stuck at max	Controller thinks temp is max, turns heater off permanently	7	3	5	105	Range check on ADC value
MCU	Watchdog timeout	System resets, brief control loss	5	2	2	20	Safe state on reset
Heater relay	Stuck closed	Uncontrolled heating, fire risk	10	2	6	120	Thermal fuse, redundant temp sensor
Firmware	Stack overflow	Undefined behaviour, crash	8	4	7	224	Stack canary, FreeRTOS stack check
Power supply	Brownout	MCU resets or corrupts data	6	3	4	72	BOR circuit, EEPROM write verify

8.3 Reliability and Availability

Definition 8.2: Reliability

Reliability $R(t)$ is the probability that a system performs its intended function without failure for a specified time t under stated conditions. For constant failure rate λ:

$$R(t) = e^{-\lambda t}$$

Key metrics:

$$\text{MTTF} = \frac{1}{\lambda} \qquad \text{(Mean Time To Failure, non-repairable)} \tag{8.1}$$

$$\text{MTBF} = \text{MTTF} + \text{MTTR} \qquad \text{(Mean Time Between Failures)} \tag{8.2}$$

$$\text{Availability} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}} \qquad \text{(Steady-state)} \tag{8.3}$$

$$\text{FIT rate} = \frac{10^9}{\text{MTTF (hours)}} \qquad \text{(Failures In Time per } 10^9 \text{ hours)} \tag{8.4}$$

8.3.1 Bathtub Curve

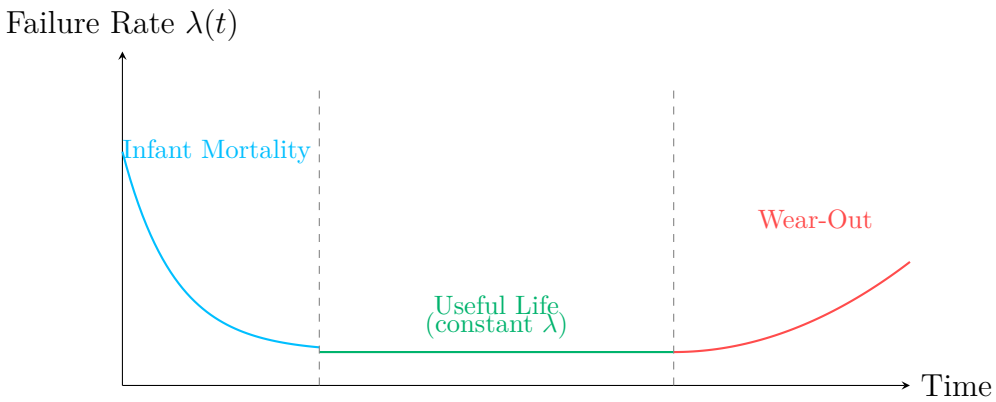

Figure 8.1: Bathtub curve: failure rate over the lifetime of electronic systems.

8.3.2 Redundancy: Triple Modular Redundancy (TMR)

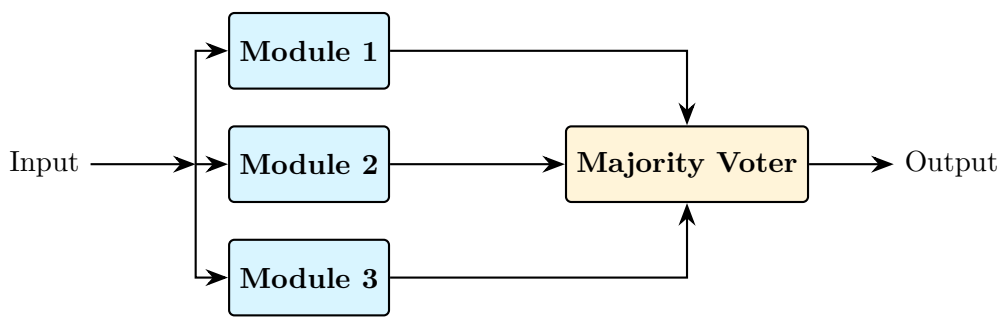

Figure 8.2: Triple Modular Redundancy: three identical modules feed a majority voter.

TMR reliability (assuming independent failures, perfect voter):

$$R_{\text{TMR}}(t) = 3R(t)^2 - 2R(t)^3 = 3e^{-2\lambda t} - 2e^{-3\lambda t}$$

TMR is more reliable than a single module for $R(t) > 0.5$ (i.e., early in the system's life).

8.4 Formal Verification

Definition 8.3: Formal Verification

Formal verification uses mathematical techniques to prove or disprove that a system design satisfies a formal specification. Unlike testing, which can only show the presence of bugs, formal verification can prove their *absence* (within the scope of the model).

8.4.1 Model Checking with SPIN and Promela

SPIN (Simple Promela Interpreter) is a model checker for verifying properties of concurrent systems.

```
/* Promela: producer-consumer with semaphores */
#define N 4 /* buffer size */

byte buffer[N];
byte in = 0, out = 0, count = 0;

/* Semaphores modeled as byte variables */
byte mutex = 1;
byte empty_slots = N;
byte full_slots = 0;

inline P(sem) { atomic { sem > 0 -> sem-- } }
inline V(sem) { atomic { sem++ } }

active proctype Producer() {
    byte item = 0;
    do
    :: P(empty_slots);
       P(mutex);
       buffer[in] = item;
       in = (in + 1) % N;
       count++;
       V(mutex);
       V(full_slots);
       item++;
    od
}

active proctype Consumer() {
    byte data;
    do
    :: P(full_slots);
       P(mutex);
       data = buffer[out];
       out = (out + 1) % N;
       count--;
       V(mutex);
       V(empty_slots);
    od
}

/* Safety property: buffer never overflows or underflows */
ltl safe { [](count >= 0 && count <= N) }
```

Listing 8.3: Promela model of a producer-consumer system with bounded buffer.

8.4.2 Temporal Logic

Temporal logic expresses properties about system behaviour over time:

Linear Temporal Logic (LTL):

- $\Box p$ ("always p"): p holds in every state.
- $\Diamond p$ ("eventually p"): p holds in some future state.
- $p \, \mathcal{U} \, q$ ("p until q"): p holds until q becomes true.
- $\bigcirc p$ ("next p"): p holds in the next state.

Common embedded system properties expressed in LTL:

- **Safety** (bad things never happen): $\Box\neg\text{overflow}$
- **Liveness** (good things eventually happen): $\Box(\text{request} \rightarrow \Diamond\text{grant})$
- **Mutual exclusion:** $\Box\neg(\text{cs}_1 \wedge \text{cs}_2)$
- **Deadlock freedom:** $\Box\Diamond\text{progress}$

8.4.3 Timed Automata and UPPAAL

UPPAAL models real-time systems as networks of timed automata with clock constraints. Useful for verifying timing properties like "the airbag deploys within 10 ms of impact detection."

8.5 MISRA-C Guidelines

Definition 8.4: MISRA-C

MISRA-C[2] is a set of software development guidelines for the C language in safety-critical and security-critical embedded systems. Originally developed for the automotive industry, it is now used across aerospace, medical, rail, and defence.

MISRA-C 2012 rule categories:

- **Mandatory:** Must be followed (e.g., no undefined behaviour).
- **Required:** Must be followed unless a formal deviation is documented.
- **Advisory:** Recommended best practice.

Selected important rules:

Table 8.3: Selected MISRA-C 2012 rules relevant to embedded programming.

Rule	Category	Description
Rule 1.3	Required	There shall be no occurrence of undefined or critical unspecified behaviour.
Rule 11.3	Required	A cast shall not be performed between a pointer to object type and a pointer to a different object type.
Rule 14.3	Required	Controlling expressions shall not be invariant.
Rule 17.7	Required	The value returned by a function having non-void return type shall be used.
Rule 21.3	Required	The memory allocation and deallocation functions of `<stdlib.h>` shall not be used.
Dir 4.1	Required	Run-time failures shall be minimised.
Dir 4.6	Advisory	`typedefs` that indicate size and signedness should be used in place of the basic numerical types.

△ Key Point: Why No Dynamic Memory?

Rule 21.3 prohibits `malloc()`/`free()` because:

1. Heap fragmentation can cause allocation failures in long-running systems.
2. Non-deterministic execution time of `malloc()`.
3. Memory leaks are catastrophic in systems that cannot be rebooted.

Instead, use statically allocated buffers, memory pools, or stack allocation.

8.5.1 CRC for Data Integrity

Cyclic Redundancy Check (CRC) is widely used for detecting data corruption in communication and storage.

```c
#include <stdint.h>

static uint32_t crc32_table[256];

void crc32_init(void) {
    for (uint32_t i = 0; i < 256; i++) {
        uint32_t crc = i;
        for (int j = 0; j < 8; j++) {
            if (crc & 1)
                crc = (crc >> 1) ^ 0xEDB88320;
            else
                crc >>= 1;
        }
        crc32_table[i] = crc;
    }
}

```

```c
uint32_t crc32_compute(const uint8_t *data, uint32_t length) {
    uint32_t crc = 0xFFFFFFFF;
    for (uint32_t i = 0; i < length; i++) {
        uint8_t index = (uint8_t)(crc ^ data[i]);
        crc = (crc >> 8) ^ crc32_table[index];
    }
    return crc ^ 0xFFFFFFFF;
}
```

Listing 8.4: CRC-32 implementation (IEEE 802.3 polynomial).

8.6 Worked Examples

Example 8.1: Reliability Calculation

A system has three components in series: MCU ($\lambda_1 = 10$ FIT), EEPROM ($\lambda_2 = 5$ FIT), and power supply ($\lambda_3 = 20$ FIT). Calculate: (a) system MTTF, (b) reliability at $t = 10{,}000$ hours, (c) availability if MTTR = 2 hours.

Solution

(a) Series system failure rate: $\lambda_{\text{sys}} = \lambda_1 + \lambda_2 + \lambda_3 = 35$ FIT.
MTTF $= \frac{10^9}{\lambda_{\text{sys}}} = \frac{10^9}{35} = 28{,}571{,}429$ hours $\approx 3{,}261$ years.
(b) $R(10{,}000) = e^{-\lambda_{\text{sys}} \times t} = e^{-35 \times 10^{-9} \times 10{,}000} = e^{-3.5 \times 10^{-4}} \approx 0.99965$ (99.965%).
(c) $A = \frac{\text{MTTF}}{\text{MTTF}+\text{MTTR}} = \frac{28{,}571{,}429}{28{,}571{,}429+2} \approx 0.99999993$ ($\sim$7 nines).

Example 8.2: TMR vs. Simplex

Compare the reliability of a simplex system vs. TMR at $t = 100{,}000$ hours if each module has $\lambda = 100$ FIT.

Solution

$\lambda = 100 \times 10^{-9}$ failures/hour. At $t = 100{,}000$ hours:
Simplex: $R_S(t) = e^{-\lambda t} = e^{-100 \times 10^{-9} \times 10^5} = e^{-0.01} = 0.99005$.
TMR: $R_{\text{TMR}}(t) = 3e^{-2\lambda t} - 2e^{-3\lambda t} = 3e^{-0.02} - 2e^{-0.03}$.
$= 3(0.98020) - 2(0.97045) = 2.94060 - 1.94090 = 0.99970$.
TMR reliability $0.99970 > 0.99005$ simplex. TMR improves reliability from 2 nines to nearly 4 nines at this operating point.

Example 8.3: FMEA Risk Prioritisation

Given the FMEA table in Table 8.2, identify the highest-risk item and propose additional mitigation measures.

Solution

Highest RPN: **Firmware stack overflow** (RPN = 224).
This is the highest risk because:

- Severity is high (8): undefined behaviour can be catastrophic.
- Occurrence is moderate (4): stack overflows are common during development.
- Detection is poor (7): often manifests as seemingly random crashes.

Additional mitigations:

1. Use MPU (Memory Protection Unit) to mark stack guard regions.
2. Enable FreeRTOS `configCHECK_FOR_STACK_OVERFLOW` (method 2).
3. Run static analysis (e.g., StackAnalyzer) to compute maximum stack depth.
4. Place stack canary values and check periodically.
5. During testing, fill stacks with a known pattern and measure high-water mark.

After mitigation, re-rate: Detection drops to 3, new RPN $= 8 \times 4 \times 3 = 96$.

Example 8.4: Formal Verification Property

Express the following requirements as LTL formulas: (a) "The motor is never on while the safety guard is open." (b) "Every sensor reading request is eventually served." (c) "The system never enters both heating and cooling modes simultaneously."

Solution

(a) $\Box(\text{guard_open} \rightarrow \neg\text{motor_on})$
(b) $\Box(\text{sensor_request} \rightarrow \Diamond\text{sensor_served})$
(c) $\Box\neg(\text{heating} \wedge \text{cooling})$
To verify these in SPIN, they would be written as:

```
/* (a) */ ltl safe_guard { [](guard_open -> !motor_on) }
/* (b) */ ltl liveness { [](sensor_request -> <>sensor_served) }
/* (c) */ ltl exclusive { [](!(heating && cooling)) }
```

8.7 Practice Problems

Exercise 8.1: Performance Profiling

You measure a function at 2,000 cycles on Cortex-M4 at 168 MHz. The function is called every 1 ms. (a) What is the CPU utilisation due to this function? (b) If you optimise it to 800 cycles using a lookup table that consumes 4 KB of Flash, discuss the trade-off.

Exercise 8.2: Reliability Series-Parallel

Design a system with two subsystems: Subsystem A ($\lambda_A = 50$ FIT) needs 99.99% reliability over 5 years. (a) Does a single module meet this? (b) Design a TMR configuration and calculate its reliability. (c) At what operating time does TMR become less reliable than simplex?

Exercise 8.3: FMEA Practice

Create an FMEA table for a drone flight controller with at least 5 failure modes. Include: IMU sensor failure, GPS loss, motor ESC failure, battery voltage drop, and communication link loss. Assign S, O, D ratings and calculate RPNs.

Exercise 8.4: Model Checking

Write a Promela model for two tasks sharing a critical section protected by Peterson's algorithm. Define an LTL property for mutual exclusion and describe how SPIN would verify it.

Exercise 8.5: MISRA-C Review

Identify MISRA-C 2012 violations in the following code and explain how to fix each:

```c
int* ptr = (int*)malloc(sizeof(int) *10);
for (int i = 0; i <= 10; i++) // potential off-by-one
    ptr[i] = i;
printf("Done");            // return value unused
free(ptr);
```

Exercise 8.6: CRC Application

A 64-byte data packet is protected by CRC-32. (a) What is the total frame size? (b) If a single bit is flipped in the data, will CRC-32 detect it? (c) What is the probability of an undetected error for a random error pattern?

Chapter 9

Modeling of Embedded Systems

Module: 9 **Lecture Hours:** 4 **ILOs Addressed:** ILO-7, ILO-8

Learning Outcomes: By the end of this module, students will be able to:

1. Draw UML diagrams (class, sequence, state machine) for embedded systems.
2. Design finite state machines (FSMs) and hierarchical state machines (HSMs).
3. Implement FSMs in C using switch-case and function pointer table approaches.
4. Model reactive systems using state flow diagrams.
5. Discuss model-driven development tools (Simulink/Stateflow, Rhapsody).

9.1 UML for Embedded Systems

The Unified Modeling Language (UML) provides standardised diagrams for documenting software architecture and behaviour. Embedded systems commonly use:

- **Class diagrams:** Show structure (modules, data types, relationships).
- **Sequence diagrams:** Show interaction (message flow between objects over time).
- **State machine diagrams:** Show behaviour (states and transitions).
- **Activity diagrams:** Show workflow/algorithms.

9.1.1 UML Class Diagram

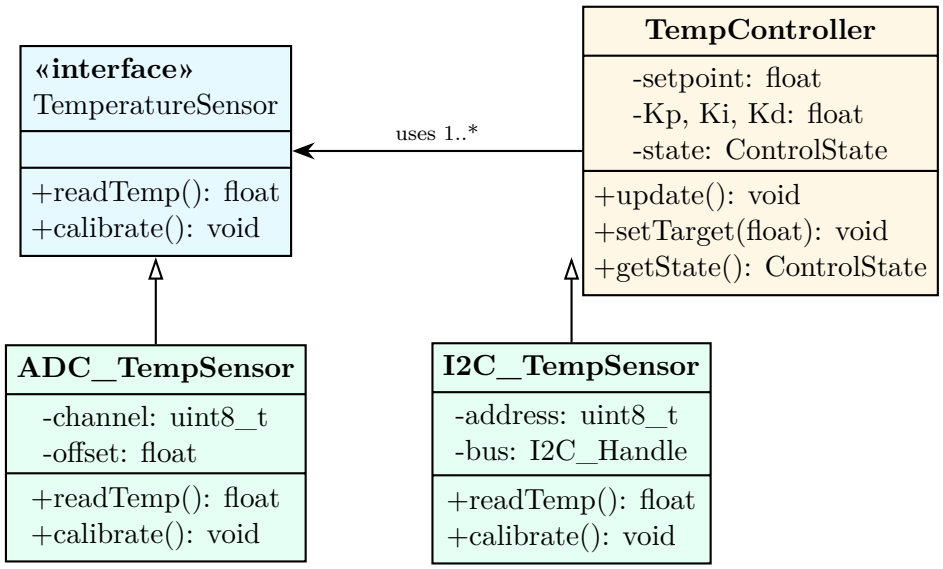

Figure 9.1: UML class diagram: temperature controller with sensor abstraction.

9.1.2 UML Sequence Diagram

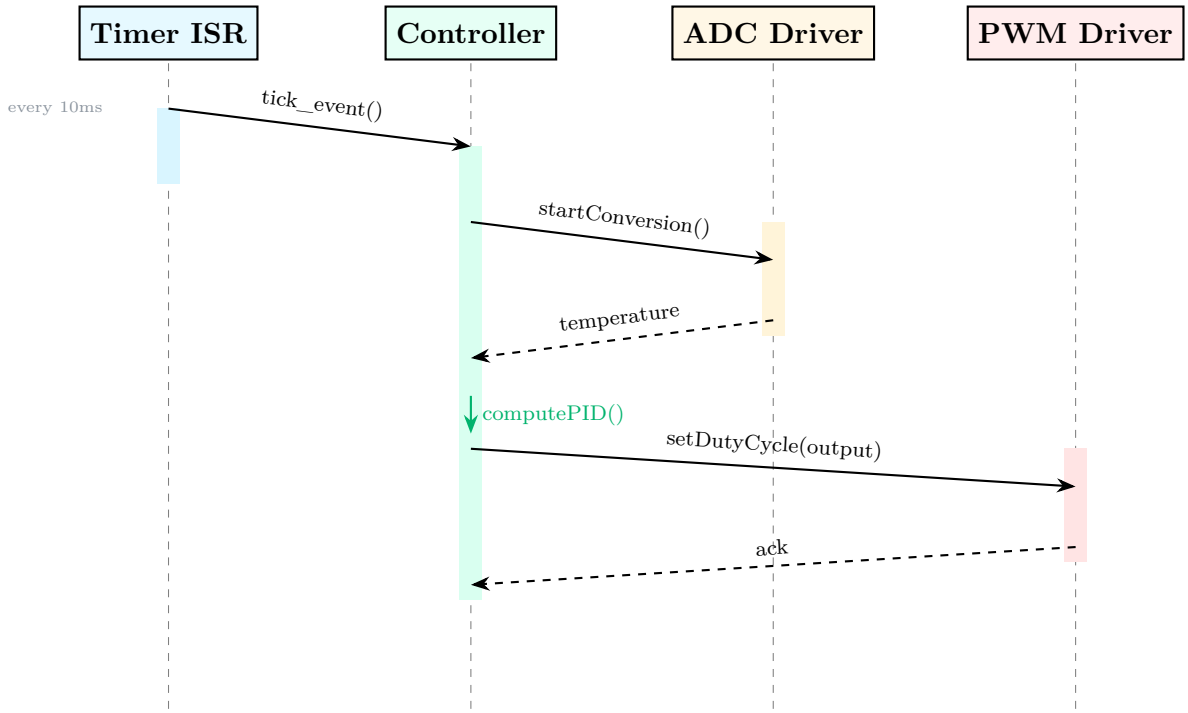

Figure 9.2: UML sequence diagram: periodic temperature control loop.

9.2 Finite State Machines (FSMs)

Definition 9.1: Finite State Machine

A finite state machine is a mathematical model of computation defined by a tuple $(S, \Sigma, \delta, s_0, F)$:

- S: finite set of states
- Σ: finite set of input events (alphabet)
- $\delta : S \times \Sigma \to S$: transition function
- $s_0 \in S$: initial state
- $F \subseteq S$: set of accepting/final states (optional for embedded FSMs)

9.2.1 Mealy vs. Moore Machines

- **Moore machine:** Output depends only on the current state. Actions are associated with states (entry/exit actions).
- **Mealy machine:** Output depends on the current state *and* the input event. Actions are associated with transitions.

9.2.2 Example: Traffic Light Controller FSM

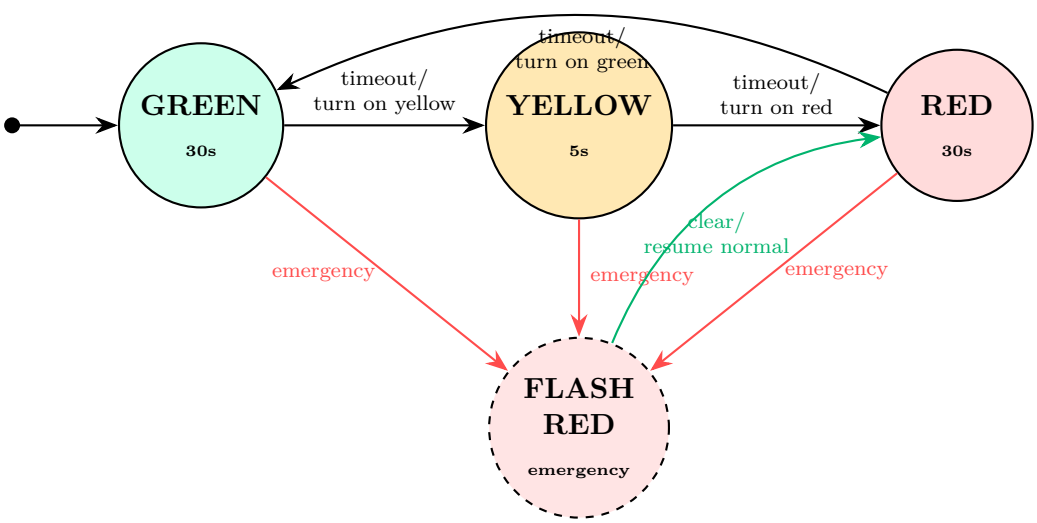

Figure 9.3: FSM for a traffic light controller with emergency override.

9.3 FSM Implementation in C

9.3.1 Switch-Case Implementation

The most common and straightforward approach:

```c
typedef enum {
    STATE_GREEN,
    STATE_YELLOW,
    STATE_RED,
    STATE_FLASH_RED
} traffic_state_t;

typedef enum {
    EVT_TIMEOUT,
    EVT_EMERGENCY,
    EVT_CLEAR
} traffic_event_t;

static traffic_state_t current_state = STATE_GREEN;

void traffic_fsm(traffic_event_t event) {
    switch (current_state) {
        case STATE_GREEN:
            if (event == EVT_TIMEOUT) {
                set_light(YELLOW);
                start_timer(5000); // 5 seconds
                current_state = STATE_YELLOW;
            } else if (event == EVT_EMERGENCY) {
                set_light(FLASH_RED);
                current_state = STATE_FLASH_RED;
            }
            break;

        case STATE_YELLOW:
            if (event == EVT_TIMEOUT) {
                set_light(RED);
                start_timer(30000); // 30 seconds
                current_state = STATE_RED;
            } else if (event == EVT_EMERGENCY) {
                set_light(FLASH_RED);
                current_state = STATE_FLASH_RED;
            }
            break;

        case STATE_RED:
            if (event == EVT_TIMEOUT) {
                set_light(GREEN);
                start_timer(30000);
                current_state = STATE_GREEN;
            } else if (event == EVT_EMERGENCY) {
                set_light(FLASH_RED);
                current_state = STATE_FLASH_RED;
```

```
            }
            break;

        case STATE_FLASH_RED:
            if (event == EVT_CLEAR) {
                set_light(RED);
                start_timer(30000);
                current_state = STATE_RED;
            }
            break;
    }
}
```

Listing 9.1: Switch-case FSM implementation for the traffic light controller.

Pros: Simple, easy to understand, compiler can optimise well.

Cons: Does not scale well for many states/events; no separation of state logic from transition logic.

9.3.2 Function Pointer Table Implementation

A table-driven approach that separates the FSM structure from the action logic:

```
#include <stdint.h>

typedef enum { ST_GREEN, ST_YELLOW, ST_RED, ST_FLASH, ST_COUNT } state_t;
typedef enum { EV_TIMEOUT, EV_EMERGENCY, EV_CLEAR, EV_COUNT } event_t;

typedef struct {
    state_t next_state;
    void (*action)(void);
} transition_t;

// Action functions
static void act_to_yellow(void) { set_light(YELLOW); start_timer(5000); }
static void act_to_red(void)  { set_light(RED);     start_timer(30000); }
static void act_to_green(void) { set_light(GREEN);  start_timer(30000); }
static void act_to_flash(void) { set_light(FLASH_RED); stop_timer(); }
static void act_none(void)    { /* no action */ }

// Transition table: [current_state][event] -> {next_state, action}
static const transition_t fsm_table[ST_COUNT][EV_COUNT] = {
    /* ST_GREEN */ {
        [EV_TIMEOUT]  = { ST_YELLOW, act_to_yellow },
        [EV_EMERGENCY] = { ST_FLASH, act_to_flash },
        [EV_CLEAR]    = { ST_GREEN, act_none     },
    },
    /* ST_YELLOW */ {
        [EV_TIMEOUT]  = { ST_RED,   act_to_red   },
```

```
        [EV_EMERGENCY] = { ST_FLASH, act_to_flash },
        [EV_CLEAR]    = { ST_YELLOW, act_none    },
    },
    /* ST_RED   */ {
        [EV_TIMEOUT]  = { ST_GREEN, act_to_green },
        [EV_EMERGENCY] = { ST_FLASH, act_to_flash },
        [EV_CLEAR]    = { ST_RED,   act_none     },
    },
    /* ST_FLASH */ {
        [EV_TIMEOUT]  = { ST_FLASH, act_none     },
        [EV_EMERGENCY] = { ST_FLASH, act_none    },
        [EV_CLEAR]    = { ST_RED,   act_to_red   },
    },
};

static state_t current_state = ST_GREEN;

void fsm_dispatch(event_t event) {
    const transition_t *t = &fsm_table[current_state][event];
    if (t->action != NULL) {
        t->action();
    }
    current_state = t->next_state;
}
```

Listing 9.2: Table-driven FSM using function pointers.

Pros: Clean separation; adding states/events only requires adding table entries; data-driven design.

Cons: Slightly more RAM/Flash for the table; all transition functions must have the same signature.

9.4 Hierarchical State Machines (HSMs)

Definition 9.2: Hierarchical State Machine

An HSM (also called a statechart, Harel statechart) extends flat FSMs with:

- **Nested states:** A superstate contains substates. Transitions common to all substates can be defined once at the superstate level.
- **Entry/exit actions:** Actions executed when entering or leaving a state.
- **History states:** Remember the last active substate when re-entering a superstate.
- **Orthogonal regions:** Concurrent substates within a superstate.

9.4.1 Statechart Example: Motor Controller

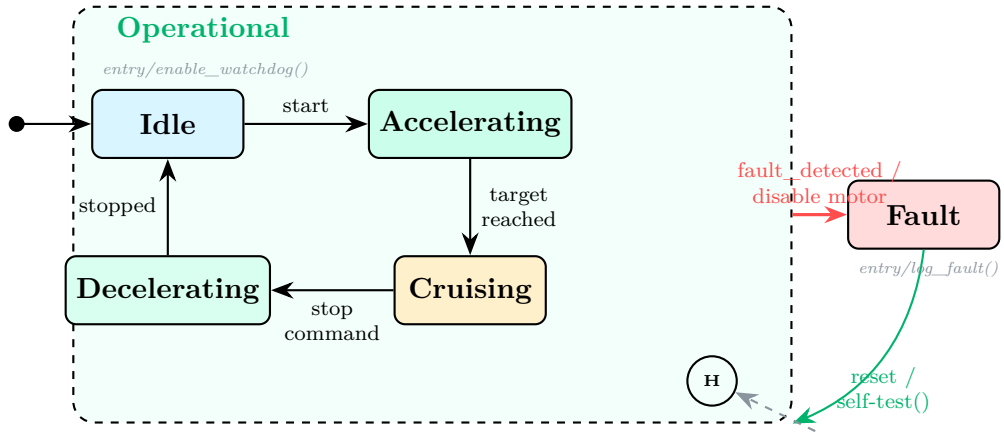

Figure 9.4: HSM: Motor controller with "Operational" superstate and "Fault" handling. The fault transition applies to all substates simultaneously.

> ◇ ***Insight***
>
> *Without hierarchy, the fault transition would need to be duplicated for every substate (Idle, Accelerating, Cruising, Decelerating). The superstate defines it once.*

9.4.2 HSM Implementation Approach

HSMs can be implemented using frameworks like QP (Quantum Platform) by Miro Samek, which provides:

- State handler functions (one per state, processes events and returns transition targets)
- Automatic entry/exit action execution during transitions
- History tracking

Simplified C pattern for nested states:

```c
typedef enum { SIG_START, SIG_STOP, SIG_FAULT, SIG_RESET } signal_t;

typedef struct {
    void (*state)(signal_t sig); // Current state handler
} hsm_t;

static hsm_t motor_hsm;

/* Superstate handler */
void operational(signal_t sig) {
    switch (sig) {
        case SIG_FAULT:
            log_fault();
            disable_motor();
            motor_hsm.state = fault_state; // Transition to Fault
```

```
            break;
        default:
            break; // Unhandled events stay in current substate
    }
}

/* Substate: Idle */
void idle_state(signal_t sig) {
    switch (sig) {
        case SIG_START:
            enable_watchdog();
            set_motor_target(target_speed);
            motor_hsm.state = accel_state;
            break;
        default:
            operational(sig); // Delegate to superstate
            break;
    }
}

/* Substate: Accelerating */
void accel_state(signal_t sig) {
    switch (sig) {
        case SIG_STOP:
            motor_hsm.state = decel_state;
            break;
        default:
            operational(sig); // Delegate to superstate
            break;
    }
}

/* Fault state (not a substate of Operational) */
void fault_state(signal_t sig) {
    switch (sig) {
        case SIG_RESET:
            if (self_test_pass()) {
                motor_hsm.state = idle_state; // Or restore history
            }
            break;
        default:
            break;
    }
}
```

Listing 9.3: Simplified hierarchical state handler pattern.

The key pattern is **delegation**: unhandled events in a substate are forwarded to the superstate handler.

9.5 State Flow Models

9.5.1 Stateflow in Simulink

MATLAB/Simulink's Stateflow tool allows graphical design of state machines with:

- Chart-level and state-level data
- Condition actions (evaluated during transitions)
- Temporal logic operators (`after(5, sec)`, `count(event)`)
- Code generation (C/C++) from the model

This model-driven approach is widely used in automotive (ISO 26262) and aerospace (DO-178C) for safety-critical development.

9.5.2 Model-Driven Development Workflow

1. **Model:** Design the system using UML/statecharts in a modelling tool.
2. **Simulate:** Run the model to validate behaviour before writing any code.
3. **Generate:** Automatically generate C/C++ code from the validated model.
4. **Verify:** Test the generated code against the model (back-to-back testing).
5. **Deploy:** Compile and flash the generated code to the target MCU.

▶ Practical Note

In industry, tools like IBM Rhapsody, MATLAB Simulink/Stateflow, and Yakindu Statechart Tools are used for model-driven development. The generated code is traceable to requirements, which is essential for certification in safety-critical domains.

9.6 Worked Examples

Example 9.1: Vending Machine FSM

Design an FSM for a simple vending machine that accepts 5-cent and 10-cent coins for a 15-cent item. Draw the state diagram and implement it using a switch-case in C.

Solution

States: {IDLE (0 cents), FIVE (5 cents), TEN (10 cents)}
Events: {COIN_5, COIN_10}
Actions: dispense(), return_change()

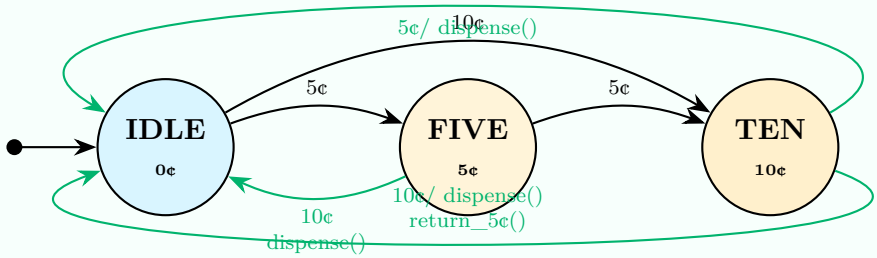

Figure 9.5: Vending machine FSM.

```c
typedef enum { ST_IDLE, ST_FIVE, ST_TEN } vm_state_t;
typedef enum { EV_COIN5, EV_COIN10 } vm_event_t;

static vm_state_t state = ST_IDLE;

void vending_fsm(vm_event_t event) {
    switch (state) {
        case ST_IDLE:
            if (event == EV_COIN5)    state = ST_FIVE;
            else if (event == EV_COIN10) state = ST_TEN;
            break;
        case ST_FIVE:
            if (event == EV_COIN5)    state = ST_TEN;
            else if (event == EV_COIN10) {
                dispense();           // 5+10 = 15
                state = ST_IDLE;
            }
            break;
        case ST_TEN:
            if (event == EV_COIN5) {
                dispense();           // 10+5 = 15
                state = ST_IDLE;
            } else if (event == EV_COIN10) {
                dispense();           // 10+10 = 20 > 15
                return_change(5);     // Return 5 cents
                state = ST_IDLE;
            }
            break;
    }
}
```

Listing 9.4: Vending machine FSM in C.

Example 9.2: UML State Machine for a Microwave Oven

Design a UML state machine diagram for a microwave oven with the following requirements: (a) The door can be opened or closed. (b) Cooking only starts when the door is closed and the start button is pressed. (c) Opening the door during cooking immediately stops the magnetron. (d) A timer counts down during cooking.

Solution

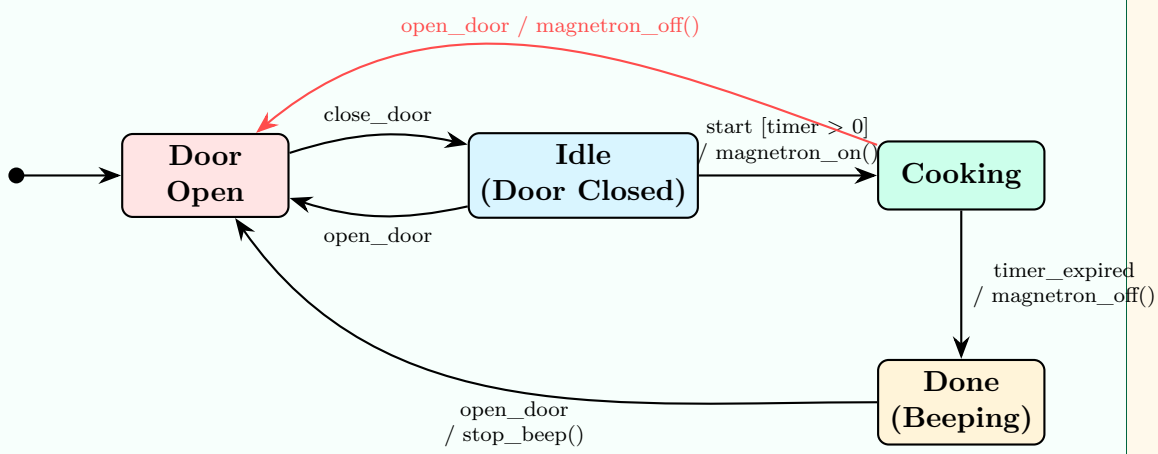

Figure 9.6: Microwave oven UML state machine diagram.

The safety-critical property: $\square(\text{door_open} \rightarrow \neg\text{magnetron_on})$ — the magnetron is **never** on when the door is open. This is enforced by the transition from Cooking to DoorOpen which calls `magnetron_off()`.

Example 9.3: Table-Driven FSM for Serial Protocol

Design a function-pointer table FSM for parsing a simple serial frame: `[STX] [LEN] [DATA...] [CRC] [ETX]`. States: WAIT_STX, READ_LEN, READ_DATA, READ_CRC, VERIFY.

Solution

```c
#define STX 0x02
#define ETX 0x03

typedef enum {
    S_WAIT_STX, S_READ_LEN, S_READ_DATA, S_READ_CRC, S_COUNT
} parser_state_t;

typedef struct {
    uint8_t buffer[256];
    uint8_t length;
    uint8_t index;
    uint8_t crc;
    parser_state_t state;
} parser_t;

typedef void (*state_handler_t)(parser_t *p, uint8_t byte);

static void handle_wait_stx(parser_t *p, uint8_t byte) {
    if (byte == STX) {
        p->index = 0;
        p->crc = 0;
        p->state = S_READ_LEN;
    }
}

static void handle_read_len(parser_t *p, uint8_t byte) {
    p->length = byte;
    p->crc ^= byte;
    p->state = (byte > 0) ? S_READ_DATA : S_READ_CRC;
}

static void handle_read_data(parser_t *p, uint8_t byte) {
    p->buffer[p->index++] = byte;
    p->crc ^= byte;
    if (p->index >= p->length) {
        p->state = S_READ_CRC;
    }
}

static void handle_read_crc(parser_t *p, uint8_t byte) {
    if (byte == p->crc) {
        process_frame(p->buffer, p->length); // Valid frame
    }
    p->state = S_WAIT_STX; // Reset for next frame
}

static const state_handler_t handlers[S_COUNT] = {
    [S_WAIT_STX] = handle_wait_stx,
    [S_READ_LEN] = handle_read_len,
    [S_READ_DATA] = handle_read_data,
    [S_READ_CRC] = handle_read_crc,
```

Example 9.4: HSM Design for Washing Machine

Sketch a hierarchical state machine for a washing machine with a "Running" superstate containing substates (Fill, Wash, Drain, Spin). A "Pause" event from any running substate should pause and remember the current phase. A "Door Open" event from any state should go to an "Error" state.

Solution

Design:

- **Superstates:** Idle, Running (with substates Fill, Wash, Drain, Spin), Paused, Error.
- **Running superstate transitions:** Pause → Paused (captures history); Door_Open → Error (safety).
- **History:** Paused stores the last active substate. Resume restores it.

The "Running" superstate handles the common Pause and Door_Open events. Each substate only handles its phase-complete event:

- Fill: water_level_reached → Wash
- Wash: wash_timer_done → Drain
- Drain: drain_complete → Spin
- Spin: spin_timer_done → Idle

The HSM eliminates 8 redundant transitions (2 events × 4 substates) that a flat FSM would require.

9.7 Practice Problems

Exercise 9.1: Elevator FSM

Design a flat FSM for a 3-floor elevator. Include states for each floor, moving up, and moving down. Show transitions for floor requests and arrival sensors. Implement the FSM using a function pointer table in C.

Exercise 9.2: UML Sequence Diagram

Draw a UML sequence diagram for an I2C transaction: Master sends a Start condition, slave address + write bit, waits for ACK, sends 2 data bytes (each with ACK), and then sends Stop. Show the timing on the lifelines.

Exercise 9.3: Mealy vs. Moore

Convert the traffic light FSM (Figure 9.3) from a Moore machine to a Mealy machine. What changes in the state diagram? Which representation would you prefer for implementation and why?

Exercise 9.4: HSM Design

Design an HSM for a smart thermostat with: (a) an "Off" state, (b) a "Heating" superstate with substates (Pre-heat, Maintain), (c) a "Cooling" superstate with substates (Pre-cool, Maintain), (d) a "Fault" state accessible from any superstate. Implement the superstate delegation pattern in C.

Exercise 9.5: Code Generation

Given the vending machine FSM from the worked example, write a Python script that reads a CSV description of the FSM (state, event, next_state, action) and generates the C switch-case implementation automatically.

Exercise 9.6: Stateflow to C

A Simulink Stateflow chart has two parallel (orthogonal) regions: a "Motor Control" region and a "Sensor Monitor" region. Explain how you would implement orthogonal regions in C without a tool like Stateflow. What are the synchronisation challenges?

References

[1] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973. DOI: 10.1145/321738.321743

[2] "MISRA-C:2012 — guidelines for the use of the C language in critical systems," Motor Industry Software Reliability Association, Standard, 2012, Third Edition, First Revision.